AI Infra M1 CUDA 编程与算子优化学习笔记：从 Warp 到 FlashAttention

> **数据来源**：本笔记的骨架、术语体系、算子优化版本的演进顺序，以及绝大部分实测数据，取自 AIInfra Guide「模块二：CUDA 编程与算子优化」的 24 篇公开文章（快速入门指南 + 第 1～8 章），访问日期 **2026-11**。
>
> **补充来源**（用于交叉核对机制、补齐原文未展开的部分，文中标注「补充」）：NVIDIA *CUDA C++ Programming Guide*（SIMT、Warp Shuffle、`cp.async`、内存一致性模型）、NVIDIA *CUDA C++ Best Practices Guide*（Occupancy、Memory Optimizations）、Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*（NeurIPS 2022）、Dao *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*（2023）、Milakov & Gimelshein *Online normalizer calculation for softmax*（2018）、Triton 官方教程 *Fused Softmax* / *Matrix Multiplication*、NVIDIA Nsight Compute 与 Nsight Systems 用户手册、NVIDIA A100/H100 架构白皮书。
>
> **标注约定**：`【补充】` = AIInfraGuide 原文没有、由上述权威来源或通用知识补齐的内容；`【推断】` = 基于材料做的推理性结论，未经原文或文档直接陈述；`【量级】` = 只保证数量级正确，不对具体尾数负责；`原文实测` = AIInfraGuide 作者在文中给出的实测值，本笔记只转述、不复现。
>
> **版权声明**：AIInfraGuide 原文版权归原作者所有，本笔记为个人学习用途的二次整理、重组与批注，不替代原文；引用原文观点与数据处已尽量保留其表述与限定条件（尤其是测试硬件 A100 80GB SXM4 这一前提）。NVIDIA 文档与白皮书、arXiv 论文、Triton 文档版权归各自权利人所有。若本文与原文冲突，以原文与官方文档为准。
>
> **一个必须先说清的口径问题**：本模块原文的所有百分比（带宽利用率、相对 cuBLAS 的达成率、TFLOPS 利用率）都是在 **A100 80GB SXM4** 上测得的。换到 H100/B200，绝对性能会变，但「哪一步优化解决哪个瓶颈」的因果链不变——这才是要背下来的东西。凡本文出现具体百分比，默认都是「原文在 A100 上的实测」，不再逐处重复。

1.1 为什么先建心智模型，而不是先写代码
1.2 SM：资源分配的最小粒度
1.3 CUDA Core vs Tensor Core：两条完全不同的算力曲线
1.4 存储层次：2～3 个数量级的落差
1.5 Memory Wall 与 Roofline：把「慢」归因到唯一一处
1.5.1 算术强度
1.5.2 三种「墙」的判别
1.6 本章小结
2.1 三级层次与它对应的硬件
2.2 内置变量与索引计算
2.2.1 一维：最常用的映射
2.2.2 二维：矩阵的标准写法
2.2.3 把线性 index 拆回坐标
2.3 kernel 启动语法与配置
2.3.1 向上取整的 grid 计算
2.3.2 Grid-Stride Loop：不随 n 变的 grid
2.4 Host/Device 内存管理
2.4.1 五件套

2.4.2 统一内存的代价
2.5 错误检查：异步模型的必付成本
2.6 本章小结
3.1 五种内存与选型决策
3.2 合并访问：一个 Warp 的 32 个请求怎么合并成事务
3.2.1 事务粒度
3.2.2 一条判据，胜过十条经验
3.2.3 AoS vs SoA
3.2.4 对齐
3.3 共享内存：Bank 编址与冲突
3.3.1 编址规则
3.3.2 冲突判定表
3.3.3 经典案例：32×32 转置的 32-way 冲突
3.3.4 解法一：Padding（+1 列）
3.3.5 解法二：XOR Swizzle（不浪费空间）
3.3.6 解法三：改线程映射（GEMM 的 Z-order）
3.4 向量化访存：float4
3.4.1 收益来自哪里
3.4.2 类型与对齐
3.4.3 尾部处理的标准写法
3.5 双缓冲：把「等数据」变成「边算边等」
3.5.1 为什么需要
3.5.2 两级双缓冲（GEMM 的标准结构）
3.5.3 `cp.async`：Ampere 之后的正确做法
3.6 本章小结
4.1 Warp 是硬件行为，不是语法
4.2 SIMT vs SIMD：一句话区别
4.3 分支发散：量化与四种规避
4.3.1 量化
4.3.2 四种规避策略
4.4 Warp Scheduler 与延迟隐藏的算术
4.5 Warp Shuffle：寄存器级通信
4.5.1 四个原语
4.5.2 相比 Shared Memory 的优势
4.5.3 蝶形归约的展开
4.5.4 `width` 参数：子 Warp 归约
4.6 本章小结
5.1 定义与「停车场」模型
5.2 三大资源限制
5.2.1 寄存器
5.2.2 共享内存
5.2.3 Block 大小
5.2.4 架构对照
5.3 手算一遍：完整例题
5.4 反直觉的核心：低 occupancy 也能快
5.4.1 什么时候 occupancy 重要
5.4.2 盲目追高的三种反噬
5.5 调优工具与流程
5.5.1 三条命令
5.5.2 代码里控制
5.6 本章小结
6.1 同步层级总览
6.2 `__syncthreads()` 的两条铁律
6.2.1 屏障的真实代价
6.3 `__syncwarp()`：Volta 之后必需
6.4 Memory Fence：可见性而不是顺序
6.5 原子操作：清单与代价
6.5.1 代价取决于冲突度
6.5.2 `atomicCAS` 是万能底座
6.6 四种原子优化策略
6.7 本章小结
7.1 为什么 Reduce 是最好的入门算子
7.2 版本速查表
7.3 V0：朴素树形，15%
7.3.1 问题一：Warp Divergence
7.3.2 问题二：Bank Conflict
7.4 V1：strided index，32%
7.5 V2：步长反转，40%（本模块最重要的一个技巧）
7.5.1 为什么同时解决两个问题
7.6 V3：每线程 2 元素，45%
7.7 V4：展开最后一个 Warp，52%
7.8 V5：模板参数全展开，62%
7.9 V6：Warp Shuffle 两级规约，72%
7.10 V7：`float4` + Grid Stride，85%
7.11 二次归约与工程落地

7.12 本章小结
8.1 为什么 GEMM 值得单独一章
8.2 Naive：0.25 的算术强度
8.3 第一级：Block Tiling（25%～40%）
8.3.1 为什么沿 K 切
8.3.2 算术强度的推导
8.3.3 协作加载要保证合并
8.4 第二级：Thread Tiling（50%～65%）——外积的威力
8.4.1 从内积到外积
8.4.2 收益量化
8.4.3 为什么是 8×8 而不是 16×16
8.5 第三级：Warp Tiling（80%～87%）
8.5.1 为什么需要
8.5.2 形状怎么选：`4×8` 优于 `1×32`
8.5.3 坐标计算（背下来）
8.6 第四级：`float4` 向量化（60%～75%）
8.6.1 A 要转置存
8.6.2 坐标公式要跟着改
8.7 第五级：Bank Conflict 消除（70%～80%）
8.7.1 float4 下冲突只在 quarter-warp 内判定
8.7.2 三种方案
8.7.3 Z-order 为什么是 4 个
8.8 第六级：双缓冲（90%～97%）
8.9 第七级：`cp.async` 与 Hopper
8.10 参数表与「写法陷阱」
8.10.1 推荐参数
8.10.2 四个「等价但慢 30%」的写法
8.11 本章小结
9.1 两篇文章解决两个不同问题
9.2 数值溢出：为什么必须减最大值
9.3 三遍扫描与它的成本
9.4 Safe Softmax 的五版演进
9.4.1 V0 为什么只有 5%
9.4.2 V1/V2：把一行拆给整个 Block
9.4.3 V4：两遍融合的边界
9.5 Online Softmax：用乘法修正代替重新计算
9.5.1 递推公式
9.5.2 数值稳定性没被牺牲
9.6 并行化 Online Softmax：合并算子要满足结合律
9.6.1 一个真实的 NaN 陷阱
9.7 Online Softmax 的五版演进
9.7.1 V3：真正的一遍（寄存器当缓存）
9.7.2 V4：Grid Stride 的适用边界
9.8 算子融合：Softmax 的终局
9.9 本章小结
10.1 标准 Attention 的两个瓶颈
10.2 FlashAttention V1：Tiling + Online Softmax
10.2.1 两个技术缺一不可
10.2.2 块大小怎么定
10.2.3 前向的五个步骤
10.2.4 V1 的循环结构（也是它的病根）
10.2.5 V1 的并行度瓶颈（最关键的一条）
10.3 IO 复杂度：为什么说它「渐近最优」
10.3.1 标准 Attention
10.3.2 FlashAttention
10.3.3 显存收益
10.4 反向传播：用重计算换显存
10.5 FlashAttention V2：三个改动 + 一个附加
10.5.1 V1 只跑到 25%～40%
10.5.2 改动一：换循环顺序（外 Q 内 KV）
10.5.3 改动二：延迟 rescaling（减非 GEMM 运算）
10.5.4 改动三：Warp 沿 Q 切（不沿 KV 切）
10.5.5 附加：Causal 块级跳过
10.5.6 保守量：存 logsumexp
10.5.7 V1 vs V2 总表
10.5.8 实测（A100，原文表）
10.6 V3、Flash-Decoding、PagedAttention（原文仅给概述）
10.6.1 FlashAttention-3（Hopper 专属）
10.6.2 Flash-Decoding：decode 阶段并行度不足
10.6.3 PagedAttention：KV Cache 的分页管理
10.7 本章小结
11.1 为什么需要「不手写 CUDA」
11.2 Triton 的 block-level 编程模型

11.2.1 心智模型对照
11.2.2 六个核心 API
11.2.3 Fused Softmax 骨架
11.3 torch.compile：Dynamo + Inductor
11.3.1 Graph Break：编译的最大敌人
11.4 TVM / XLA 的定位
11.5 本章小结
12.1 三个工具的分工
12.2 Nsight Systems：宏观时序
12.3 Nsight Compute：Kernel 级下钻
12.3.1 SOL（Speed of Light）面板怎么读
12.3.2 stall 原因对照动作
12.4 PyTorch Profiler：框架层对账
12.5 本章小结
13.1 三条「迁移学习」的捷径
13.2 一个反例：不要照搬 CPU 的直觉
14.1 复习清单（按「能不能上手写」排序）
14.2 自测题（12 题，答案在后）
14.3 答案
14.4 三条「如果只记三件事」
15.1 AIInfraGuide 模块二（24 篇，访问日期 2026-11）
15.2 补充来源
15.3 免责与口径重申

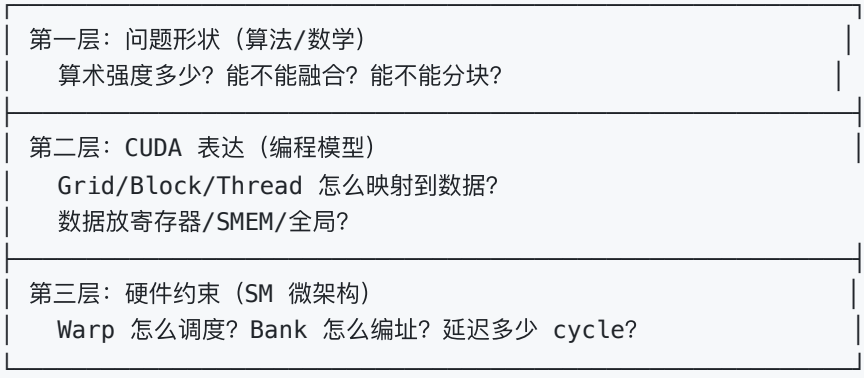
第 1 章 GPU 硬件与 CUDA 心智模型

1.1 为什么先建心智模型，而不是先写代码

CUDA 的语法一天能学完：`__global__`、`<<<grid, block>>>`、`threadIdx.x`。但「为什么我的 kernel 只有峰值带宽的 15%」这个问题，语法给不了答案。AIInfraGuide 模块二的组织方式很值得学：它没有按语法点铺开，而是**按瓶颈类型铺开**——第 1 章把程序跑起来，第 2 章把四类瓶颈（分支、访存、占用率、同步）各讲一遍，第 3～6 章用四个真实算子把这四类瓶颈各撞一遍，第 7～8 章讲「不手写」和「怎么测」。

所以读这个模块的正确姿势是：**先有一张「性能是被什么限制的」地图，再往里填语法。**

这张地图只有三层：



绝大多数「优化」发生在第一层和第二层；第三层的作用是**判定你到底撞到了哪堵墙**，以及**为什么某一层优化到此为止**。

1.2 SM：资源分配的最小粒度

GPU 芯片的层级是 `GPC → TPC → SM → Core`。对写 kernel 的人，只需要盯住 **SM（Streaming Multiprocessor）** 这一层，因为它是**资源分配的最小单位**：一个 Thread Block 被整体调度到一个 SM 上，Block 内所有线程共享这个 SM 的寄存器堆、共享内存和 L1。

以 H100 的 SM 为例（与前置模块的硬件章保持同一口径）：

组件	数量（H100 每 SM）	作用	写 kernel 时的意义
FP32 CUDA Core	128	标量浮点/整数运算	FFMA 的吞吐上限
Tensor Core（第四代）	4	矩阵乘累加（MMA）	想吃满算力必须走它
Warp Scheduler	4	每周期各选 1 个就绪 Warp 发射	决定「同周期最多几条指令」
Load/Store Unit	32	访存指令执行	访存指令排队的地方
SFU	16	`expf` / `sinf` 等超越函数	Softmax 的隐性瓶颈
Register File	256 KB（65536 × 32-bit）	线程私有寄存器	Occupancy 的头号限制因素
Shared Memory / L1	228 KB（可配置）	片上可编程 SRAM + 自动缓存	Tiling 的物理基础

两个立刻能用的推论：

1. **一个 SM 有 4 个 Warp Scheduler，每周期最多发射 4 条 Warp 指令**。所以「SM 里塞满 64 个 Warp」并不等于「64 倍并行」，硬件每周期只能发 4 条。
2. **寄存器堆 65536 个 32-bit 寄存器**是个硬数字。一个 Block 用 256 线程 × 128 寄存器 = 32768，那么一个 SM 最多放 2 个这样的 Block——这就是 GEMM 里「25% occupancy 却很快」的算术来源（第 5、8 章）。

1.3 CUDA Core vs Tensor Core：两条完全不同的算力曲线

维度	CUDA Core	Tensor Core
执行的操作	标量 FMA（`a*b+c`，一次 2 FLOP）	小矩阵乘加（如 16×8×16 的 MMA）
编程方式	普通 C 表达式	`wmma` / `mma.sync` / `wgmma` /CUTLASS/Triton `tl.dot`
数据来源	寄存器	寄存器（Ampere）或 Shared Memory（Hopper `wgmma`）
对齐要求	无	矩阵维度通常需 8/16 的倍数

维度	CUDA Core	Tensor Core
峰值（H100 SXM，稠密）	FP32 约 67 TFLOPS	FP16/BF16 约 989 TFLOPS，FP8 约 1979 TFLOPS

这张表解释了一个反复出现的现象：**一个 kernel 的「FLOPs 达成率」低，可能不是代码写得差，而是它压根没走 Tensor Core。** GEMM 优化阶梯的最后一级（第 8 章）就是「换硬件单元」，而不是「继续抠指令」。

【补充】NVIDIA 产品页上的 Tensor Core 数字常带星号表示含 2:4 稀疏加速，读规格时必须确认是否标注 "with sparsity"；本笔记统一采用稠密口径（稀疏值约为稠密的 2 倍）。

1.4 存储层次：2～3 个数量级的落差

同一份数据放在不同层级，延迟差 2～3 个数量级，带宽差 1～2 个数量级。这是全部优化技术的**唯一物理依据**。

层级	作用域	容量（H100 口径）	延迟	带宽	谁来管理
Register	线程私有	每线程 ≤255 个	~0 cycle	最高	编译器
Shared Memory	Block 内	228 KB/SM	~30 cycle	~31 TB/s【量级】	程序员
L1 Cache	SM 内	与 SMEM 共用物理 SRAM	~30 cycle	与 SMEM 共享	硬件
L2 Cache	全卡	约 50 MB	~200 cycle	~9.45 TB/s【量级】	硬件
HBM（全局内存）	全局	80 GB	~600 cycle	3.35 TB/s	程序员
Host 内存	跨 PCIe	—	~μs 级	PCIe 5.0 ×16 约 64 GB/s	程序员

三个容易记错、但很关键的点：

- **L1 和 Shared Memory 是同一块物理 SRAM**。所以「SMEM 用得多」会挤掉 L1，二者不是两份独立预算。
- **SMEM 的 ~31 TB/s 是理论估算**（32 bank × 4 B/cycle × 132 SM × ~1.83 GHz），前提是**零 Bank Conflict**。一旦冲突，这个数字除以冲突路数。
- **HBM 的 3.35 TB/s 是官方理论峰值**。Reduce 这类纯访存 kernel 能到 85%～90% 就已经是「贴顶」了（第 7 章）。

1.5 Memory Wall 与 Roofline：把「慢」归因到唯一一处

1.5.1 算术强度

```
算术强度 AI = 浮点运算量（FLOPs） / 访存字节数（Bytes）
脊点（ridge point）= 峰值算力 / 峰值带宽
AI < 脊点  ⇒  Memory Bound（等数据）
AI > 脊点  ⇒  Compute Bound（等计算）
```

三个常被引用的脊点（原文表，BF16 Tensor Core 口径）：

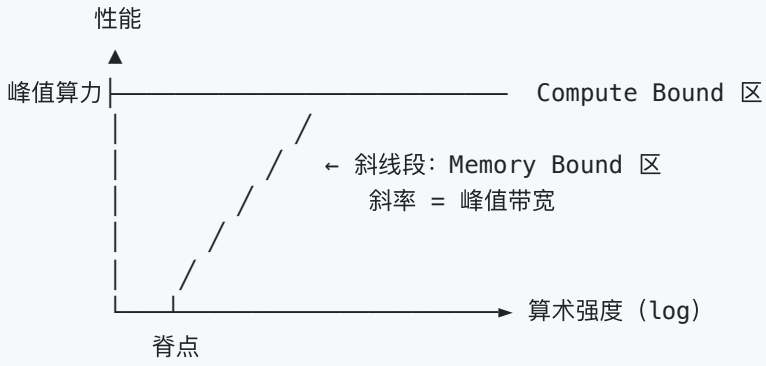
系统	算力（TFLOP/s）	带宽（TB/s）	脊点（FLOP/Byte）
A100 80GB SXM / HBM2e	~312	~2	~156
H100 SXM / HBM3	~989	~3.35	~295
B200 / HBM3e	~2250	~8	~281

这个数字怎么用：拿 H100 来说，每从 HBM 读 1 Byte，你得做够约 295 次 FP16 浮点操作，算力才可能跑满。而绝大多数算子离这个数差着 2～3 个数量级：

算子	算术强度（推算）	结论
向量加法 `c=a+b`	1 FLOP / 12 B ≈ **0.083**	极端 Memory Bound
ReLU / GELU（逐元素）	~0.1～0.25	Memory Bound
Reduce（求和）	1 FLOP / 4 B = **0.25**	Memory Bound
Softmax（三遍扫描）	~0.5 左右（含 exp 的 SFU 开销另计）	Memory Bound
朴素 GEMM（无 tiling）	**0.25**	严重 Memory Bound
Tiled GEMM（BM=BN=128）	**32**	仍低于脊点，需继续分块
分块 GEMM + 寄存器复用 + Tensor Core	数百	Compute Bound

【推断】上表最后一行之所以能越过脊点，靠的不是「算术强度公式变了」，而是**分块把 HBM 流量降到了 Tiling 级别**：同一个 BM×BN tile 的 K 维数据被整个 Block 复用，HBM 流量从 O(MNK) 降到 O(MN·K/BK) 量级。Roofline 里的 Bytes 指的是**HBM 字节**，不是 SMEM 字节——混淆这两者是最常见的分析错误。

1.5.2 三种「墙」的判别



现象（Nsight Compute 视角）	归因	该做什么
Memory Throughput 高（>70%）、Compute 低	Memory Bound	减访存：融合、Tiling、向量化、提高复用
Compute Throughput 高、Memory 低	Compute Bound	换单元（Tensor Core）、减指令、提 ILP
两者都不高，但 stall 很多	Latency Bound	提 Occupancy 或提 ILP（第 5 章）
两者都不高，且 Warp 数很少	并行度不足	改 grid 划分（第 10 章 FA V2 的核心）

这张表是整份笔记的「总开关」：后面每一章的优化手段，都可以回溯到这张表的某一行。

1.6 本章小结

记住这一条	因为它决定了
SM 是资源分配单位，Block 整体进出	为什么「差一点」的资源会导致 occupancy 断崖
Warp 是调度单位，32 线程锁步	为什么分支发散和 bank conflict 都按 32 计
存储层次差 2~3 个数量级	为什么 Tiling 几乎万能
算术强度对比脊点	判断 Memory Bound 还是 Compute Bound
每 SM 4 个 Scheduler	为什么 occupancy 有天花板，且不是越高越好

第 2 章 CUDA 编程模型：Grid、Block、Thread 与启动配置

2.1 三级层次与它对应的硬件

逻辑概念	硬件对应	能否同步	能否共享内存
Grid（一次 launch 的全部线程）	整个 GPU	仅 kernel 边界（或 cooperative launch）	否
Block（线程块）	一个 SM	是（`__syncthreads()`）	是（同一 Block 内）
Thread（线程）	CUDA Core 上的 lane	—	寄存器私有
Warp（32 线程）	**SM 的调度单元**	`__syncwarp()`	寄存器可跨 lane 读（Shuffle）

原文用「学校—班级—学生」类比 Grid/Block/Thread。这个类比要补一句才完整：**班级（Block）一定在同一栋楼（SM）里上课，但同一栋楼可以同时容纳好几个班级**；而「小组」（Warp）是老师点名的单位，小组内 32 人必须做同一个动作。

2.2 内置变量与索引计算

变量	类型	含义
`threadIdx`	`dim3`	线程在 Block 内的坐标
`blockIdx`	`dim3`	Block 在 Grid 内的坐标
`blockDim`	`dim3`	每个 Block 的线程数
`gridDim`	`dim3`	Grid 的 Block 数
`warpSize`	`int`	恒为 32（不要硬编码 32，用它）

2.2.1 一维：最常用的映射

```
int gid = blockIdx.x * blockDim.x + threadIdx.x;   // 全局线程号
if (gid < n) { ... }                                // 边界检查，必写
```

```
gridDim.x = 4, blockDim.x = 8

Block 0: [ 0  1  2  3  4  5  6  7]
Block 1: [ 8  9 10 11 12 13 14 15]
Block 2: [16 17 18 19 20 21 22 23]
Block 3: [24 25 26 27 28 29 30 31]
```

2.2.2 二维：矩阵的标准写法

```
int col = blockIdx.x * blockDim.x + threadIdx.x;   // x 走列（连续方向）
int row = blockIdx.y * blockDim.y + threadIdx.y;   // y 走行
if (row < M && col < N) {
    int linear = row * N + col;                    // 行主序线性化
}
```

为什么 x 给列而不是给行：行主序下同一行的元素在内存里连续，`threadIdx.x` 是 Warp 内变化最快的维度，让 x 沿列走才能让「Warp 内连续线程访问连续地址」——这就是合并访问（第 3 章）。写反了（x 给行）会得到 stride=N 的跨步访问，带宽直接掉一个数量级。

2.2.3 把线性 index 拆回坐标

在 GEMM/Reduce 里经常反过来用：拿到一个线性 tid，要拆成 (row, col) 去做线程重排。

```
// 慢：非 2 的幂时，除法和取模编译成多条指令
int r = tid / 8, c = tid % 8;

// 快：除数是 2 的幂时，编译器直接给移位和掩码
int r = tid >> 3, c = tid & 7;
```

【补充】原文把这条列为「被忽视的性能杀手」之一：仅靠这类写法优化（位运算、消除冗余循环、`#pragma unroll` 生效、模板常量折叠），参考实现就能从约 60% cuBLAS 提到约 93% cuBLAS。原因不是算术变快了，而是**整数/分支指令占用了本该给 FFMA 的发射槽**。

2.3 kernel 启动语法与配置

```
kernel<<<gridDim, blockDim, sharedMemBytes, stream>>>(args...);
```

参数	类型	说明
`gridDim`	`dim3` 或 `int`	启动多少个 Block
`blockDim`	`dim3` 或 `int`	每个 Block 多少线程
`sharedMemBytes`	`size_t`	动态共享内存字节数（默认 0）
`stream`	`cudaStream_t`	执行流（默认 0）

2.3.1 向上取整的 grid 计算

```
int block = 256;
int grid  = (n + block - 1) / block;   // 整数向上取整
kernel<<<grid, block>>>(d_in, d_out, n);
```

这个式子是所有「最后一个 Block 有越界线程」问题的来源，所以 kernel 内必须有 `if (idx < n)`。反过来，如果先 `n = grid * block` 补齐（用 `cudaMemset` 填 0），就能省掉边界判断——Reduce 里常用这个技巧换掉每个线程一次的比较分支（第 7 章 V3 的写法）。

2.3.2 Grid-Stride Loop：不随 n 变的 grid

```
__global__ void saxpy(float a, const float* x, float* y, int n) {
    int stride = blockDim.x * gridDim.x;              // 总线程数
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride) {
        y[i] = a * x[i] + y[i];
    }
}
```

三个好处：

1. **grid 大小可以按「刚好填满 GPU」来设**（如 `SM 数 × 4`），而不是被 n 绑架；
2. 天然支持任意 n，包括超过 grid 维度上限（x 维 $2^{31}-1$）的巨数组；
3. 每个线程的固定开销（启动、索引计算）被摊到多次迭代上。

【推断】这也是为什么 Reduce 的 V7 和 Online Softmax 的 V4 都收敛到 grid-stride：它不是某个算子的技巧，而是**「让 grid 只和硬件有关，让循环只和数据有关」**这条工程原则的直接体现。

2.4 Host/Device 内存管理

2.4.1 五件套

```
float *d_a;
size_t bytes = n * sizeof(float);

cudaMalloc(&d_a, bytes);                                    // 分配
cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);        // H2D
kernel<<<grid, block>>>(d_a, n);                             // 计算
cudaMemcpy(h_a, d_a, bytes, cudaMemcpyDeviceToHost);        // D2H
cudaFree(d_a);                                               // 释放
```

API	用途	坑
`cudaMalloc`	分配设备内存	返回 256 B 对齐的指针（向量化的前提）
`cudaMallocHost`	锁页内存	传输快 2～3 倍，但**吃主机内存、不可过量**
`cudaMallocManaged`	统一内存	首次访问触发缺页迁移，性能敏感场景别用
`cudaMemcpyAsync`	异步拷贝	必须配 `cudaMallocHost` 才能真异步
`cudaMemset`	清零	常用来「补齐 n」消除边界判断

2.4.2 统一内存的代价

```
float* data;
cudaMallocManaged(&data, bytes);
// CPU 初始化 → GPU kernel → CPU 读取，同一个指针
cudaMemPrefetchAsync(data, bytes, device_id);    // 显式预取，避免缺页抖动
```

适合	不适合
原型、快速迭代	追求极致性能的生产代码
链表/树等指针结构	大块单向传输（显式 `cudaMemcpy` 更快）
不确定 GPU 访问哪些数据	明确的输入→计算→输出模式

2.5 错误检查：异步模型的必付成本

kernel launch 是异步的，主机端拿到的返回值只代表「参数和队列没问题」，不代表 kernel 跑对了。所以标准动作是**三段检查**：

```
#define CUDA_CHECK(call) do {                                          \
    cudaError_t err = call;                                            \
    if (err != cudaSuccess) {                                          \
        fprintf(stderr, "CUDA error at %s:%d: %s\n",                   \
                __FILE__, __LINE__, cudaGetErrorString(err));          \
        exit(EXIT_FAILURE);                                            \
    }                                                                  \
} while (0)

// 1) API 调用
CUDA_CHECK(cudaMalloc(&d_a, bytes));

// 2) kernel 启动（抓「参数非法/共享内存超限/架构不匹配」）
myKernel<<<grid, block>>>(d_a, n);
CUDA_CHECK(cudaGetLastError());

// 3) 同步等待（抓「越界访问/非法指令」这类运行时错误）
CUDA_CHECK(cudaDeviceSynchronize());
```

陷阱	表现	正解
只查 `cudaMalloc` 不查 launch	程序「静默」跑出全 0 结果	每次都查 `cudaGetLastError()`
用 `cudaGetLastError()` 查运行期错误	查不到（错误发生在 kernel 内部）	再加 `cudaDeviceSynchronize()`
在计时区间里放了 `cudaDeviceSynchronize()`	测出来是「包含同步等待的墙钟」	用 CUDA event 或 profiler 计时
release 版关掉了检查	越界访问变成「偶尔对」	用 `compute-sanitizer` 跑一遍

【补充】`compute-sanitizer`（原 `cuda-memcheck`）是定位越界/竞态的标准工具，比加 `printf` 快得多：

```
compute-sanitizer --tool memcheck ./my_app
compute-sanitizer --tool racecheck ./my_app     # 共享内存竞态
```

2.6 本章小结

要点	一句话
索引	`gid = blockIdx.x*blockDim.x + threadIdx.x`，x 走连续维度
边界	`(n + block - 1)/block` 配 `if (idx < n)`，或补齐 n 省掉判断
遍历	grid-stride loop 让 grid 只和硬件有关
计时	CUDA event 包 kernel，换算成有效带宽
查错	API / launch / 同步 三段式，再加 sanitizer

第 3 章 内存模型与访问优化：从合并访问到双缓冲

3.1 五种内存与选型决策

内存	声明方式	作用域	生命周期	典型用途
寄存器	局部变量	线程	线程	累加器、tile 片段
共享内存	`__shared__` / `extern __shared__`	Block	Block	Tiling、规约中间结果
全局内存	`cudaMalloc`	全局	应用	输入输出张量
常量内存	`__constant__`	全局只读	应用	卷积核、系数表（≤64 KB）
局部内存	编译器自动（溢出）	线程	线程	**应尽量避免**，本质在显存

选型决策树（原文给的版本，很好用）：

```
数据在 kernel 里会被改吗?
├── 只读
│    ├── 所有线程读同一个值，且 ≤64KB ──→ 常量内存
│    └── 否则 ──→ 全局内存 + __ldg()
└── 读写
     ├── 需要 Block 内共享 ──→ 共享内存 + __syncthreads()
     └── 不需要 ──→ 全局内存（保证合并访问）
```

「局部内存」要单独提一句：它名字里没有「global」，但物理上就在显存里。触发条件是**编译期无法确定索引的数组**（如 `float buf[N]` 且 `N` 是变量、或用寄存器数组下标动态索引）。这就是为什么「寄存器数组 + 动态下标」是 CUDA 里最贵的写法之一。

3.2 合并访问：一个 Warp 的 32 个请求怎么合并成事务

3.2.1 事务粒度

全局内存以 **32 B sector** 为基本事务粒度，L1 cache line 是 128 B（= 4 个 sector）。一个 Warp 的 32 个线程各读 1 个 float（4 B）= 128 B：

```
完美合并：
  Thread 0..31 → 地址 0,4,8,...,124  → 落在同一 128B line 的 4 个 sector
  → 1 条 cache line 请求，4 个 sector 全用上，利用率 100%

跨步 2（stride=2）：
  Thread 0..31 → 地址 0,8,16,...,248 → 跨越 2 条 128B line（256B）
  → 2 条 line 请求，但每条只用一半 → 有效带宽 50%

随机：
  Thread 0..31 → 32 个互不相邻的地址 → 最多 32 个 sector 请求
  → 有效带宽 ~3%
```

原文给的对照表（A100，读 1 GB）：

访问模式	事务数	有效带宽利用率	相对耗时
连续对齐（stride=1）	1x	~95%	1x
stride=2	2x	~50%	2x
stride=4	4x	~25%	4x
stride=32	32x	~3%	32x
完全随机	32x	~3%	32x

3.2.2 一条判据，胜过十条经验

> **同一个 Warp 内，`threadIdx` 相邻的线程应该访问地址相邻的元素。**

这条判据能推出几乎所有结论：

场景	是否符合	改法
`data[gid]`	✅	—
`data[gid * 2]`（隔一个取一个）	❌ 2x 事务	改数据布局，或先连续读再筛选
行主序矩阵按列遍历	❌ stride=N	转置 / Tiling / 交换 x–y
`AoS` 结构体取单个字段	❌ stride=sizeof(struct)	改 `SoA`

场景	是否符合	改法
每个线程读同一行但不同列（SMEM tile）	✅（若 tile 行内连续）	—

3.2.3 AoS vs SoA

```
// ❌ AoS：相邻线程间隔 16 B（4 个 float）
struct Particle { float x, y, z, w; };
Particle p[N];
float v = p[tid].x;

// ✅ SoA：相邻线程间隔 4 B
struct Particles { float x[N], y[N], z[N], w[N]; };
float v = P.x[tid];
```

【补充】SoA 不是「GPU 专属技巧」，它和 SIMD 时代的「数组结构体拆分」是同一个思路：**让「同一字段的连续元素」在内存里连续**，这样向量化和合并访问才能同时成立。

3.2.4 对齐

`cudaMalloc` 保证 256 B 对齐，所以「从 0 开始连续访问」天然对齐。**真正会出事的是手动偏移**：

```
float* base = ...;              // 256B 对齐
float4* v = (float4*)(base + 3); // 偏移 12B，不满足 16B 对齐 → 未定义行为/慢路径

// ✅ 安全做法：头部标量处理，主体向量化
int head = (4 - (offset & 3)) & 3;
```

3.3 共享内存：Bank 编址与冲突

3.3.1 编址规则

```
Bank = (字节地址 / 4) % 32
```

```
地址   0~3   → Bank 0
地址   4~7   → Bank 1
...
地址 124~127 → Bank 31
地址 128~131 → Bank 0     ← 回绕，冲突的根源
```

为什么是 32：Warp 有 32 个 lane，硬件希望「理想情况下 32 个 lane 各占一条车道，一个周期全过」。所以 32 个 bank 不是随便定的，是**为 Warp 宽度定制**的。

3.3.2 冲突判定表

情形	是否冲突	原因
32 线程访问 32 个不同 bank	无	理想，1 次事务
多线程访问**同一 bank 的同一地址**	无	硬件广播（broadcast）
2 线程访问同一 bank 的**不同地址**	2-way	串行 2 次
N 线程访问同一 bank 的不同地址	N-way	吞吐降为 1/N

⚠️ 只有「同 bank、不同地址」才冲突；「同地址」是免费的广播。另外 **bank conflict 只在 Warp 内定义**，跨 Warp 不存在冲突（它们本来就分时执行）。

3.3.3 经典案例：32×32 转置的 32-way 冲突

```
__shared__ float tile[32][32];

// 写：连续线程写同一行的不同列 → 无冲突
tile[threadIdx.y][threadIdx.x] = in[gy * N + gx];

// 读：连续线程读同一列的不同行
out[gx * N + gy] = tile[threadIdx.x][threadIdx.y];
//   tid.x=0 → tile[0][ty] → 地址 (0*32+ty)*4  → Bank = ty%32
//   tid.x=1 → tile[1][ty] → 地址 (1*32+ty)*4  → Bank = (32+ty)%32 = ty%32
//   tid.x=2 → tile[2][ty] → Bank = (64+ty)%32 = ty%32
//   ⇒ 32 个线程全部落在同一个 bank 的不同地址 = 32-way conflict
```

3.3.4 解法一：Padding（+1 列）

```
__shared__ float tile[32][32 + 1];    // 33 列
// Bank = (row*33 + col) % 32
//   row=0 → ty%32
//   row=1 → (33+ty)%32 = (1+ty)%32
//   row=2 → (66+ty)%32 = (2+ty)%32
//   ⇒ 每个线程落在不同 bank，冲突消失
```

代价是每行浪费 4 B。**注意 Padding 的副作用**：它会破坏 16 B 对齐，从而**禁用 float4 访问**。所以 GEMM 里更常用下面的 swizzle。

3.3.5 解法二：XOR Swizzle（不浪费空间）

```
// 写
tile[row][col ^ row] = val;
// 读（同样的变换）
val = tile[row][col ^ row];
```

道理：用 XOR 把「行号」的信息混进「列号」，让同一列的不同行散到不同 bank。CUTLASS 里大量使用这种「地址变换代替 padding」的手法。

3.3.6 解法三：改线程映射（GEMM 的 Z-order）

GEMM 那一章给出了更彻底的分析：**在 float4（128-bit）访存下，bank conflict 是在 quarter-warp（8 线程）内部判定的**，因为一个 Warp 的 128-bit 加载会被拆成 4 个 memory transaction。

访存宽度	每 Warp 字节数	transaction 数	冲突判定范围
32-bit（float）	128 B	1	全 Warp（32 线程）
64-bit（float2）	256 B	2	half-warp（16 线程）
128-bit（float4）	512 B	4	quarter-warp（8 线程）

于是「warp 形状」直接决定冲突。原文对 `4×8` warp 布局下两种 tile 的逐 lane 分析结论是：

布局	tileA 事务数	tileB 事务数	合计
行主序 `4×8`（基线）	2（8 路广播 + uniform 合并）	8（2-way 冲突）	10
8×8 拆 4×4 子块	2	4	6
Z-order 映射	2	2	**4**

Z-order 的 lane 映射只有两行：

```
int lane_id  = threadIdx.x % 32;
int lane_row = lane_id % 2 + (lane_id / 16) * 2;    // 4 行
int lane_col = (lane_id % 16) / 2;                  // 8 列
```

效果是「每个 quarter-warp 变成 2 行 × 4 列」，于是 tileA 的 4 路广播、tileB 的 2 路广播都能成立，再加上 `xor 1` / `xor 2` 的跨 quarter-warp 合并条件，把 10 个 transaction 压到 4 个（减少 60%）。

【推断】这一段是全模块最「硬」的内容，也是最能区分「会写 CUDA」和「懂 CUDA」的地方。它的方法论值得单独记：**先算 transaction 数，再谈优化**——而不是凭感觉调 padding。

3.4 向量化访存：float4

3.4.1 收益来自哪里

```
// 标量：4 条 LDG.32
float a = in[i], b = in[i+1], c = in[i+2], d = in[i+3];

// 向量：1 条 LDG.128
float4 v = *reinterpret_cast<const float4*>(in + i);
```

收益	机制
指令数减 75%	1 条 `LDG.128` 代替 4 条 `LDG.32`，发射槽省下来给 FFMA
事务利用率提高	每线程 16 B，Warp 一次覆盖 512 B 连续区间
ILP 提高	一条指令带 4 个独立数据，更容易填满流水线
循环开销除 4	迭代次数、地址计算、分支判断都变成 1/4

原文的实测口径：向量化通常带来 **15%～30%** 的提升（GEMM），在 Softmax/Reduce 里则是「把已经优化过的版本再推高 15%～25%」。

3.4.2 类型与对齐

类型	大小	对齐要求
`float2`	8 B	8 B
`float4`	16 B	16 B
`int4`	16 B	16 B
`double2`	16 B	16 B

3.4.3 尾部处理的标准写法

```
__global__ void copy_vec(const float* __restrict__ in, float* __restrict__ out, int n) {
    int i = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
    if (i + 3 < n) {
        *reinterpret_cast<float4*>(out + i) = *reinterpret_cast<const float4*>(in + i);
    } else {
        for (int j = i; j < n; ++j) out[j] = in[j];   // 尾巴标量处理
    }
}
```

⚠ 三个必查项：① 指针 16 B 对齐；② `n % 4` 的尾巴；③ 用 `__restrict__` 告诉编译器「不重叠」，它才敢重排加载。

3.5 双缓冲：把「等数据」变成「边算边等」

3.5.1 为什么需要

全局内存延迟约 300～600 cycle。如果代码是「加载 → 同步 → 计算 → 同步 → 加载」，那么计算单元在每个加载阶段完全空闲：

```
无流水线（串行）：
  时间 →  [LDG 600cy][FFMA][LDG 600cy][FFMA]
  计算单元    空闲        忙      空闲       忙       ← 利用率 ~50%

双缓冲（重叠）：
  时间 →  [LDG buf1        ][LDG buf0        ]
          [FFMA buf0][FFMA buf1]              ← 加载与计算完全重叠
```

3.5.2 两级双缓冲（GEMM 的标准结构）

```
第 1 级：Global → Shared 双缓冲
  __shared__ float As[2][BK][BM];
  __shared__ float Bs[2][BK][BN];

第 2 级：Shared → Register 双缓冲
  float a_frag[2][TM];
  float b_frag[2][TN];
```

主循环的指令顺序是**关键**：

```
┌──────────────────────────────────────────────┐
│ 1. 发射 LDG（把下一轮数据读到寄存器暂存）          │
│ 2. 执行 FFMA（用当前 buffer 的数据）              │
│ 3. 执行 STS（把暂存寄存器写进另一个 buffer）       │
│ 4. __syncthreads() 切换 buffer                 │
└──────────────────────────────────────────────┘
```

📌 先发 LDG 再算 FFMA 是重点：LDG 是异步的，发射后线程继续执行 FFMA；等 FFMA 算完，LDG 的数据通常已经到位。**这样 300+ cycle 的延迟被计算完全遮住**，而不是被动地「多开 Warp 硬扛」。

3.5.3 `cp.async`：Ampere 之后的正确做法

```
// 直接从 global 拷到 shared，不经过寄存器中转
cp.async.cg.shared.global [smem_ptr], [gmem_ptr], 16;
cp.async.commit_group;
cp.async.wait_group N;      // 最多还有 N 组没完成
__syncthreads();
```

对比项	LDG + STS	`cp.async`
寄存器中转	需要（约 8 个/线程）	不需要
流水线深度	受寄存器数量限制	可做 3～4 stage
同步粒度	必须整块 `__syncthreads()`	`commit/wait_group` 可分组
适用架构	全部	sm_80+

C++ 层封装：

```
#include <cuda/pipeline>
__shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 2> state;
auto pipe = cuda::make_pipeline(group, &state);
pipe.producer_acquire();
cuda::memcpy_async(group, smem, gmem, bytes, pipe);
pipe.producer_commit();
pipe.consumer_wait();
// ... 计算 ...
pipe.consumer_release();
```

Hopper 进一步把这件事交给硬件：**TMA**（Tensor Memory Accelerator）做多维张量的地址计算与搬运，`wgmma` 直接读 Shared Memory 做矩阵乘——搬运和计算彻底解耦。

3.6 本章小结

技术	解决什么	代价
合并访问	全局内存事务数	需要改数据布局
对齐	额外事务	需要处理头部/尾部
padding / swizzle	SMEM bank conflict	空间或索引复杂度
向量化 `float4`	指令数、ILP	对齐与尾部处理
双缓冲 / `cp.async`	访存延迟	共享内存与寄存器翻倍
`__ldg` / cache hint	缓存污染	需要确定只读

一句话总结这一章：**内存优化的本质是「用更少的、更整的、更早的」访问，换更多的计算时间。** 更少（复用/融合）、更整（合并/向量化）、更早（双缓冲/预取）。

第 4 章 Warp 执行模型：SIMT、分支发散与延迟隐藏

4.1 Warp 是硬件行为，不是语法

一个 Block 被分配到 SM 后，硬件按 `threadIdx` 顺序每 32 个切一个 Warp：

Block 内 tid	Warp
0～31	Warp 0
32～63	Warp 1

Block 内 tid	Warp
64～95	Warp 2

两个直接后果：

1. **`blockDim` 必须是 32 的倍数**，否则最后一个 Warp 有线程空转（如 100 线程 → 4 个 Warp，最后 4 个 lane 干活、28 个陪跑）。
2. **Warp 编号可以用位运算算**（GEMM 里天天用）：`warp_id = tid >> 5`，`lane_id = tid & 31`。

4.2 SIMT vs SIMD：一句话区别

维度	SIMD（AVX-512）	SIMT（CUDA Warp）
编程视角	显式操作向量寄存器	写标量代码，硬件广播到 32 lane
分支	用 mask 跳过 lane	硬件**串行化**两条路径
寻址	lane 通常连续	每线程可独立寻址（可 gather）
寄存器状态	一份向量寄存器	每线程独立寄存器 + 逻辑独立 PC

SIMT 的代价全在「分支」上，收益全在「可以独立寻址」上——这就是为什么 `data[tid]` 很自然，而 `if (tid % 2)` 很贵。

4.3 分支发散：量化与四种规避

4.3.1 量化

```
if (tid % 2 == 0) { data[tid] = expf(data[tid]); }   // 路径 A
else              { data[tid] = logf(data[tid]); }   // 路径 B
```

一个 Warp 里 16 个线程走 A、16 个走 B。硬件先发 A 的指令（B 的线程被 mask 掉），再发 B 的指令（A 的线程被 mask 掉）：

```
无分支：[A 指令 10 条]                          → 10 个周期
有分支：[A 指令 10 条][B 指令 10 条]            → 20 个周期（2x）
最坏：  32 个 lane 走 32 条不同路径             → 32x
```

原文的判据：**Branch Efficiency = 非发散分支数 / 总分支数**，Nsight Compute 里可以逐条分支看活跃 lane 比例，理想是 100%（32/32）。

4.3.2 四种规避策略

策略	做法	适用
分支按 Warp 对齐	`if (tid / 32 % 2)` 代替 `if (tid % 2)`	两段逻辑都很重时
算术替代分支	`y = fmaxf(x, 0.f)` 代替 `if (x>0)`	逐元素、代价小的分支
数据重排	预处理把同路径数据聚到一起	可离线预处理的数据
循环条件统一	所有线程跑相同 trip count，用 predicate 决定是否写	尾部处理

循环展开那条尤其值得记：

```
// ❌ 每线程退出时机不同 → 每次迭代都可能发散
for (int i = tid; i < n; i += stride) { ... }

// ✅ 固定 trip count，用 predicate 决定「算不算」
int trips = (n + stride - 1) / stride;
for (int t = 0; t < trips; ++t) {
    int i = tid + t * stride;
    if (i < n) { ... }
}
```

【补充】Volta 起有 Independent Thread Scheduling（每线程独立 PC 和调用栈），它不是「消除发散」，而是让调度更细粒度、并让 `__syncwarp()` 这类 warp 内同步变得必要。**发散的代价并没有消失。**

4.4 Warp Scheduler 与延迟隐藏的算术

每个 SM 有 4 个 Warp Scheduler，每周期各挑 1 个就绪 Warp 发 1 条指令。一个 Warp 发完 `LDG` 后要等 ~400 cycle，这段时间它不可调度，调度器必须找别人。

「需要多少 Warp 才能藏住延迟」的粗略估算（原文）：

```
需要 Warp 数 ≥ 访存延迟 / 每条指令的发射间隔 = 400 / 4 = 100
但一个 SM 最多 64 个 Warp → 物理上藏不满
```

这个「藏不满」的结论很重要，它解释了为什么：

- 纯访存 kernel（Reduce、Softmax）**必须**靠提高 occupancy + 向量化 + grid-stride 三件套逼近带宽上限；
- 计算密集 kernel（GEMM）**不需要**高 occupancy，靠 ILP（每线程多条独立 FFMA 链）就能藏延迟；
- Warp 切换是**零开销**的（寄存器常驻 SM，无需保存/恢复），这是 GPU 敢用「多 Warp 轮转」代替「乱序执行」的底气。

4.5 Warp Shuffle：寄存器级通信

4.5.1 四个原语

```
T __shfl_sync     (unsigned mask, T var, int srcLane, int width = 32);
T __shfl_up_sync  (unsigned mask, T var, unsigned delta, int width = 32);
T __shfl_down_sync(unsigned mask, T var, unsigned delta, int width = 32);
T __shfl_xor_sync (unsigned mask, T var, int laneMask, int width = 32);
```

指令	源 lane	典型用途
`__shfl_sync`	`srcLane`	广播（如把 lane 0 的结果发给全 Warp）
`__shfl_up_sync`	`lane - delta`	前缀和（scan）
`__shfl_down_sync`	`lane + delta`	树形归约（最常用）
`__shfl_xor_sync`	`lane ^ laneMask`	蝶形归约、全 lane 都拿到结果

4.5.2 相比 Shared Memory 的优势

对比项	SMEM 规约	Shuffle 规约
数据路径	寄存器 → SMEM → 寄存器	寄存器 → 寄存器
Bank conflict	可能有	不存在
同步	每轮 `__syncthreads()`	Warp 内天然同步
延迟	~20～30 cycle	~5 cycle

原文给的量化对比（256 线程 Block 的 Block 级求和）：

方式	SMEM 用量	同步次数	指令数
纯 SMEM 逐步折半	1024 B	8 次 `__syncthreads()`	~64
Shuffle + SMEM（两级）	128 B（32 个 float）	1 次	~13

4.5.3 蝶形归约的展开

```
__device__ __forceinline__ float warpReduceSum(float val) {
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;                      // 只有 lane 0 拿到总和
}
```

```
初始:  [1] [2] [3] [4] [5] [6] [7] [8]      (以 8 lane 示意)
off=4: [1+5] [2+6] [3+7] [4+8] ...
off=2: [6+10] [8+12] ...
off=1: [16+20] = 36   ← lane 0
```

4.5.4 `width` 参数：子 Warp 归约

```
// 把 32 lane 当成 4 组 8 lane，各组独立归约
for (int off = 4; off > 0; off >>= 1)
    val += __shfl_down_sync(0xffffffffu, val, off, 8);
// lane 0/8/16/24 各持有本组和
```

【推断】`width` 在「一个 Warp 要处理多个独立小任务」时特别有用，比如 batch 很小的 decode、或者一行长度不足 32 的 softmax。

4.6 本章小结

概念	一句话
Warp	32 线程锁步，`blockDim` 必须是 32 的倍数
发散	分支串行化，最坏 32x；按 Warp 对齐或算术化
调度	4 Scheduler/周期，靠 Warp 数量 + ILP 藏延迟
Shuffle	寄存器直通，~5 cycle，归约首选

第 5 章 Occupancy 与资源分配：不是越高越好

5.1 定义与「停车场」模型

```
Occupancy = 每 SM 活跃 Warp 数 / 每 SM 支持的最大 Warp 数
```

A100/H100 每 SM 最大 64 Warp（2048 线程）。若只能驻留 32 Warp，occupancy = 50%。

这个指标的价值只有一个：**它衡量「有多少 Warp 可用于隐藏延迟」**。它不衡量「算得多快」。

5.2 三大资源限制

5.2.1 寄存器

```
每 SM 可驻留线程数 = floor(SM 寄存器总数 / 每线程寄存器数)
但寄存器以 256 个为一组、按 Warp 分配:
每 Warp 分配量 = ceil(每线程寄存器 × 32 / 256) × 256
```

A100（65536 寄存器/SM）：

每线程寄存器	每 Warp 实际分配	SM 可容纳 Warp	Occupancy
32	1024	64	100%
48	1536	42	~65%
64	2048	32	50%
128	4096	16	25%
255	8160 → 对齐 8192	8	12.5%

5.2.2 共享内存

```
每 SM 可驻留 Block 数 = floor(SM SMEM 总量 / 每 Block SMEM)
```

A100（164 KB 可用/SM，Block 256 线程）：

每 Block SMEM	SM 可容纳 Block	总线程	Occupancy
8 KB	20（但受最大 32 Block 限制）	5120 → 截到 2048	100%
32 KB	5	1280	62.5%
64 KB	2	512	25%
164 KB	1	256	12.5%

SMEM 的分配粒度是 128 B（部分架构 256 B），不是字节级——所以「刚好差 128 B」也会掉一档。

5.2.3 Block 大小

Block 大小	每 SM Block 上限	总 Warp	Occupancy
32（1 Warp）	32（Block 数限制）	32	50%
64（2 Warp）	32	64	100%
128（4 Warp）	16	64	100%
256（8 Warp）	8	64	100%
512（16 Warp）	4	64	100%
1024（32 Warp）	2	64	100%

32 线程的 Block 只能到 50%——**这就是「Block 不要小于 128」的算术理由**。

5.2.4 架构对照

资源	Ampere A100	Ada RTX 4090	Hopper H100
每 SM 最大线程	2048	1536	2048
每 SM 最大 Warp	64	48	64
每 SM 最大 Block	32	16	32
每 SM 寄存器	65536	65536	65536
每线程最大寄存器	255	255	255
每 SM SMEM 上限	164 KB	100 KB	228 KB
每 Block 最大 SMEM	163 KB	99 KB	227 KB
每 Block 最大线程	1024	1024	1024

5.3 手算一遍：完整例题

条件：每线程 40 寄存器、Block 256 线程（8 Warp）、每 Block 16 KB SMEM、A100。

```
步骤 1（寄存器）：40 × 32 = 1280 寄存器/Warp
                  floor(65536 / 1280) = 51 Warp
步骤 2（SMEM）   ：floor(164 KB / 16 KB) = 10 Block = 80 Warp
步骤 3（Block 数）：10 < 32，不触顶
步骤 4（线程数）  ：上限 64 Warp
取 min(51, 80, 64) = 51 Warp
但 Block 整体进出：floor(51/8) × 8 = 48 Warp
Occupancy = 48 / 64 = 75%          ← 瓶颈是寄存器
```

最后那一步「floor 到 Block 的整数倍」是很多人算错的地方：Block 不能半个进来。

5.4 反直觉的核心：低 occupancy 也能快

原文给的对照（同一 GEMM 的两个版本）：

版本	寄存器/线程	SMEM/Block	Occupancy	实测 GFLOPS
Version A	32	8 KB	100%	800
Version B	96	48 KB	33%	**1200**

B 用更多寄存器和 SMEM 换来了**更高的数据复用率**和**更充分的 ILP**，代价是 Warp 变少，但每个 Warp 有 64 条独立的 FMA 链，根本不需要靠「多 Warp 轮转」来藏延迟。

5.4.1 什么时候 occupancy 重要

该追求高 occupancy	可以接受低 occupancy
Elementwise（向量加、激活）	GEMM / 矩阵乘
归约（Reduce、Softmax 的行内规约）	FlashAttention（拿 SMEM 换 HBM）
简单 stencil	任何 Compute Bound kernel
复用率接近 0	复用率高、ILP 充分

5.4.2 盲目追高的三种反噬

反噬	机制
寄存器溢出（spill）	强压 `maxrregcount` → 变量落到 local memory（显存），慢数十倍
复用不足	为了省 SMEM 缩小 tile → 全局访存反而变多
缓存抖动	Warp 太多，争抢有限的 L1/L2，miss 率上升

5.5 调优工具与流程

5.5.1 三条命令

```
# 1) 看每个 kernel 的寄存器和 SMEM（最常用）
nvcc -Xptxas -v -arch=sm_90 my.cu -o my
#   ptxas info: Used 40 registers, 16384 bytes smem, 380 bytes cmem[0]

# 2) 全局限制寄存器（会 spill，慎用）
nvcc -maxrregcount=64 my.cu

# 3) 用 ncu 看 achieved vs theoretical
ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active,launch__occupancy,\
launch__registers_per_thread,launch__shared_mem_per_block_allocated ./my
```

5.5.2 代码里控制

```
// 提示编译器：每 Block 最多 256 线程，目标每 SM 至少 2 个 Block
__global__ void __launch_bounds__(256, 2) myKernel(...) { ... }

// 动态 SMEM 超过 48KB 必须显式申请
cudaFuncSetAttribute(myKernel,
    cudaFuncAttributeMaxDynamicSharedMemorySize, 164 * 1024);
```

`__launch_bounds__(256, 2)` 的推导链是：SM 有 65536 寄存器 → 要驻留 256×2 = 512 线程 → 每线程上限 65536/512 = 128 → 编译器按 128 压缩寄存器分配。**它本质是「用寄存器换 occupancy」的旋钮**。

5.6 本章小结

记住	因为
occupancy 是「延迟隐藏的潜力」，不是性能	只是分子分母之比
寄存器是第一限制因素	65536/SM 是硬顶，且按 Warp 粒度对齐
Block 必须整体进出	算出来要向下取整到 Block 的整数倍
GEMM 25% occupancy 正常	靠 ILP 不靠 TLP
别设「必须 75%」这种目标	用 profiler 测吞吐，不看指标本身

第 6 章 同步与原子：正确性优先，性能其次

6.1 同步层级总览

粒度	原语	语义	开销
Warp	`__syncwarp(mask)`	Warp 内到达一致	极低
Block	`__syncthreads()`	全 Block 到达 + 之前写入可见	低（等最慢线程）
Device	`__threadfence()`	写入对全 GPU 可见（不阻塞）	中
Grid	cooperative launch `grid.sync()`	全 Grid 屏障	高，要求全部 Block 常驻

6.2 `__syncthreads()` 的两条铁律

```
__shared__ float smem[256];
smem[tid] = in[gid];
__syncthreads();              // 铁律 1：写后读之间必须有
float left = smem[tid - 1];
```

铁律 1：同一 Block 内所有线程必须到达同一个 `__syncthreads()`。

```
// ❌ 死锁/未定义：一半线程永远到不了这个屏障
if (tid < 128) {
    __syncthreads();
}

// ❌ 同样危险：trip count 不同
for (int i = tid; i < n; i += stride) {
    __syncthreads();
}
```

铁律 2：屏障不保证「跨 Block」的可见性，也不保证「屏障外的写入」有序。

6.2.1 屏障的真实代价

`__syncthreads()` 本身的硬件开销只有几个周期，贵的是两件事：

1. **等最慢的线程**：分支发散或访存不均时，快线程全部空转；
2. **打断指令重排**：编译器不能在屏障两侧自由调度，流水线被打断。

所以在 Reduce 的 V4 里，「最后 5 轮循环里的 `__syncthreads()` 全是多余的」能带来 15% 的提升——不是因为屏障慢，而是因为**它挡住了后面 5 轮的流水**。

6.3 `__syncwarp()`：Volta 之后必需

```
smem[tid] = value;            // 同 Warp 内通过 SMEM 传值
__syncwarp(0xffffffffu);      // Volta+ 必须显式同步
float nbr = smem[tid ^ 1];
```

场景	Pre-Volta	Volta+
Warp 内经 SMEM 通信	隐式安全（但不规范）	必须 `__syncwarp()`
Shuffle 前后	不需要	不需要（自带同步语义）
Vote 前后	不需要	不需要

6.4 Memory Fence：可见性而不是顺序

```
__threadfence_block();    // Block 内可见
__threadfence();          // 全 device 可见
__threadfence_system();   // 含 CPU / 其他 GPU
```

fence 不阻塞线程，它只保证「fence 之前的写，在 fence 之后对目标范围可见」。经典用法是「最后一个 Block 做收尾」：

```
__device__ unsigned int counter = 0;

if (tid == 0) {
    partial[blockIdx.x] = block_sum;
    __threadfence();                                    // 先把 partial 推出去
    unsigned ticket = atomicInc(&counter, gridDim.x);  // 再抢号
    isLast = (ticket == gridDim.x - 1);
}
__syncthreads();
if (isLast && tid == 0) {
    // 此时能安全读所有 partial[]
}
```

📌 没有 `__threadfence()`，最后一个 Block 可能读到别人的**旧值**——因为写入还留在 L1 里。这是「单 Block 内测试永远正确、多 Block 偶发错误」的经典来源。

6.5 原子操作：清单与代价

操作	函数	支持类型
加	`atomicAdd`	int, unsigned, float, double
减	`atomicSub`	int, unsigned
最小/最大	`atomicMin/Max`	int, unsigned（float 需 CAS 模拟）
交换	`atomicExch`	int, unsigned, float
CAS	`atomicCAS`	int, unsigned, unsigned long long
位运算	`atomicAnd/Or/Xor`	int, unsigned
递增/递减	`atomicInc/Dec`	unsigned

6.5.1 代价取决于冲突度

场景	影响
每个线程原子操作不同地址	接近非原子
同 Warp 多线程竞争同一地址	串行化，最差 32x

场景	影响
跨 Block 大量线程竞争同一地址	极慢，可能成为全局瓶颈

延迟口径：**全局原子约 400～600 cycle**（要走到 L2/DRAM），**SMEM 原子约 20～100 cycle**。

6.5.2 `atomicCAS` 是万能底座

```
__device__ double atomicAddDouble(double* addr, double val) {
    unsigned long long* a = (unsigned long long*)addr;
    unsigned long long old = *a, assumed;
    do {
        assumed = old;
        old = atomicCAS(a, assumed,
                __double_as_longlong(__longlong_as_double(assumed) + val));
    } while (assumed != old);        // 失败说明被别人改了，重试
    return __longlong_as_double(old);
}
```

⚠️ CAS 循环在高竞争下会反复重试，这是「必须减少原子冲突」的根本原因。

6.6 四种原子优化策略

策略	做法	冲突变化
分层归约	Warp → Block → Grid 逐层聚合	N → gridDim.x
SMEM 原子代替全局原子	先在 SMEM 聚合再写全局	延迟 400cy → 20cy
私有化（privatization）	每线程/每 Warp 维护副本，最后合并	运行时冲突 → 0
Warp 聚合	Warp 内先归约，1 个线程做 1 次原子	32 → 1

分层归约的标准写法（也是 Reduce V0 的「正确解法」）：

```
__shared__ float block_sum;
if (tid == 0) block_sum = 0.f;
__syncthreads();

float val = (gid < n) ? in[gid] : 0.f;
val = warpReduceSum(val);                // 第 1 层：Warp 内 Shuffle

if (lane == 0) atomicAdd(&block_sum, val); // 第 2 层：每 Warp 1 次 SMEM 原子
__syncthreads();

if (tid == 0) atomicAdd(out, block_sum);   // 第 3 层：每 Block 1 次全局原子
```

这段代码值得背下来：它把「N 个线程竞争 1 个地址」变成「每 Block 1 次全局原子」，是第 7 章 V0→V6 的压缩版。

6.7 本章小结

原语	记住这一句
`__syncthreads()`	全 Block 必须都到；不能放条件分支里
`__syncwarp()`	Volta+ 的 Warp 内 SMEM 通信必须加
`__threadfence()`	管可见性不管顺序；跨 Block 收尾必用
原子	快慢只看冲突度；能分层就别硬怼

第 7 章 经典算子之一：Reduce 的八版演进（V0→V7）

7.1 为什么 Reduce 是最好的入门算子

Reduce 是「多输入 → 单输出」：求和、求最值、点积。它同时具备两个特征：

- **数学上极简**：只有一个加法，没有任何「算法优化」空间；
- **性能上极难**：算术强度 `1 FLOP / 4 Byte = 0.25 FLOP/Byte`，是纯粹的 Memory Bound。

所以它的每一分性能都来自「工程手段」，是检验「你会不会写 GPU 代码」的最佳标尺。原文的 8 个版本，本质上是一份**「Memory Bound kernel 优化清单」**——每版只解决一个瓶颈。

测试条件（原文）：A100 80GB SXM4，带宽 2039 GB/s，L2 40 MB，108 SM，每 SM 164 KB SMEM；数据规模 N = 2^{27}（128M float，512 MB）。

7.2 版本速查表

版本	核心优化	消除的瓶颈	带宽利用率（原文实测）	相对速度
V0	朴素树形（`tid % (2*step)==0`）	无	~15%	1.0x
V1	strided index	Warp Divergence（大部分）	~32%	2.1x
V2	步长反转（`blockDim/2` 递减）	Bank Conflict + Divergence	~40%	2.7x
V3	每线程 2 元素	空闲线程、grid 过大	~45%	3.0x

版本	核心优化	消除的瓶颈	带宽利用率（原文实测）	相对速度
V4	展开最后一个 Warp	多余的 `__syncthreads()`	~52%	3.5x
V5	模板参数全展开	循环/分支开销	~62%	4.1x
V6	Warp Shuffle 两级规约	SMEM 往返	~72%	4.8x
V7	`float4` + Grid Stride	指令数、grid 未填满	**~85%**	5.7x

📌 85% 已经接近实际可达上限：受 ECC、时钟波动影响，**Reduce 类 kernel 的目标区间就是 85%～90%**。

7.3 V0：朴素树形，15%

```
__global__ void reduce_v0(float* in, float* out, int n) {
    extern __shared__ float smem[];
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    smem[tid] = (gid < n) ? in[gid] : 0.0f;
    __syncthreads();

    for (int s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0) smem[tid] += smem[tid + s];
        __syncthreads();
    }
    if (tid == 0) out[blockIdx.x] = smem[0];
}
```

它有两个致命问题，而且**都不是「算得慢」**：

7.3.1 问题一：Warp Divergence

`tid % (2*s) == 0` 让同一 Warp 内一半线程进、一半线程不进：

```
s=1: 偶数 lane 工作 → 16/32 活跃 → 50%
s=2: 每 4 个 1 个活 → 25%
s=4: 12.5% ...
```

分支串行化，实际吞吐按活跃比例打折。

7.3.2 问题二：Bank Conflict

`s=1` 时，lane 0 读 `smem[0]`、`smem[1]`；lane 16 读 `smem[32]`、`smem[33]`——`smem[0]` 与 `smem[32]` 同属 Bank 0 → 2-way。`s=2` 时变 4-way，`s=4` 时 8-way。

V0 用一句话概括：一个 kernel 同时踩了分支和 bank conflict 两个坑，所以只有 15%。

7.4 V1：strided index，32%

把「按 tid 间隔挑选」改成「按 stride 显式算下标」：

```
for (int s = 1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;              // 每个线程算自己的下标
    if (index < blockDim.x) smem[index] += smem[index + s];
    __syncthreads();
}
```

`blockDim=256` 时逐轮看活跃线程：

s	活跃 tid 范围	完整 Warp 数	是否发散
1	0～127	4	无
2	0～63	2	无
4	0～31	1	无
8	0～15	0（半个 Warp）	仅 Warp 0 内
16/32/64/128	更少	0	每轮 1 次

前 3 轮完全没有发散，只有最后 5 轮在 Warp 0 内发散——**开销从「每轮都付」变成「只在尾部付」**，带宽 15% → 32%。

但 bank conflict 变得更严重了：`s=1` 时 lane 0 读 `smem[0]`、lane 16 读 `smem[32]` → 2-way；`s=2` 时 4-way；`s=4` 时 8-way。**V1 只解决了一半问题。**

7.5 V2：步长反转，40%（本模块最重要的一个技巧）

把规约方向**反过来**：从 `blockDim.x/2` 开始减半，始终让低编号线程活跃。

```
for (int s = blockDim.x / 2; s > 0; s >>= 1) {
    if (tid < s) smem[tid] += smem[tid + s];
    __syncthreads();
}
```

7.5.1 为什么同时解决两个问题

Bank conflict 消失：`s=128` 时，lane 0 读 `smem[0],smem[128]`，lane 1 读 `smem[1],smem[129]`，……lane 31 读 `smem[31],smem[159]`。32 个 lane 的起始地址模 32 恰好是 0～31 → **零冲突**。`s=64`、`s=32` 同理。

Divergence 消失：`tid < s` 且 s 从大到小，活跃线程始终是「前面的连续整数个」——每个 Warp 要么整体工作，要么整体空闲。

```
s=128: Warp 0~3 工作，Warp 4~7 空闲   ← 整 Warp 粒度
s=64 : Warp 0~1 工作，其余空闲
s=32 : Warp 0 工作
s=16 : Warp 0 的一半 → 开始发散（但只剩 Warp 0）
```

一个改动，两个瓶颈一起消失——这就是「理解硬件」比「试参数」值钱的地方。

7.6 V3：每线程 2 元素，45%

```
int gid = blockIdx.x * (blockDim.x * 2) + threadIdx.x;
float val = 0.f;
if (gid < n)             val  = in[gid];
if (gid + blockDim.x < n) val += in[gid + blockDim.x];
smem[tid] = val;
```

对比项	V2（1 元素/线程）	V3（2 元素/线程）
每 Block 处理元素	256	512
需要的 Block 数	524,288	262,144
加载阶段计算	0 次加法	1 次加法/线程
规约轮数	8	8（不变）

收益来自两点：**Block 数减半**（调度开销降）、**加载阶段就做了有用功**（不再是纯搬运工）。

【推断】这一步其实是在「提高每线程的字节吞吐」——每个线程读 2 个跨 `blockDim` 的元素，两次访问都是合并的（因为 Warp 内相邻线程仍然相邻），所以带宽不降、指令效率上升。

7.7 V4：展开最后一个 Warp，52%

当 `s <= 32` 时只剩 Warp 0 工作，而 **Warp 内本来就锁步**，`__syncthreads()` 全是多余的：

```
__device__ void warpReduce(volatile float* smem, int tid) {
    smem[tid] += smem[tid + 32];
    smem[tid] += smem[tid + 16];
    smem[tid] += smem[tid +  8];
    smem[tid] += smem[tid +  4];
    smem[tid] += smem[tid +  2];
    smem[tid] += smem[tid +  1];
}

// 主循环只跑到 s > 32
for (unsigned s = blockDim.x / 2; s > 32; s >>= 1) {
    if (tid < s) smem[tid] += smem[tid + s];
    __syncthreads();
}
if (tid < 32) warpReduce(smem, tid);
```

blockDim=256 时，8 轮循环变 3 轮（128/64/32），省掉 5 次屏障。

⚠️ **`volatile` 不能省**：不加的话编译器会把 `smem[tid]` 缓存进寄存器，后面的读拿到旧值。

7.8 V5：模板参数全展开，62%

```
template <int BLOCK_SIZE>
__global__ void reduce_v5(float* in, float* out, int n) {
    // ...
    if (BLOCK_SIZE >= 512) { if (tid < 256) smem[tid] += smem[tid + 256]; }
    if (BLOCK_SIZE >= 256) { if (tid < 128) smem[tid] += smem[tid + 128]; }
    if (BLOCK_SIZE >= 128) { if (tid <  64) smem[tid] += smem[tid +  64]; }
    if (tid < 32) warpReduce(smem, tid);
    if (tid == 0) out[blockIdx.x] = smem[0];
}
```

因为 `BLOCK_SIZE` 是编译期常量，`BLOCK_SIZE >= 512` 为假时**整块代码被删除**，最终只剩一条紧凑的直线代码：没有循环跳转、没有死分支、没有循环变量更新。

调用侧要按 block 大小分发：

```
switch (block_size) {
    case 512: reduce_v5<512><<<grid, 512, 512*4>>>(d_in, d_out, n); break;
    case 256: reduce_v5<256><<<grid, 256, 256*4>>>(d_in, d_out, n); break;
    case 128: reduce_v5<128><<<grid, 128, 128*4>>>(d_in, d_out, n); break;
}
```

7.9 V6：Warp Shuffle 两级规约，72%

```
__device__ float warpReduceSum(float val) {
    #pragma unroll
    for (int off = 16; off > 0; off >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, off);
    return val;
}

__global__ void reduce_v6(float* in, float* out, int n) {
    int tid  = threadIdx.x;
    int lane = tid & 31;
    int wid  = tid >> 5;
    int gid  = blockIdx.x * (blockDim.x * 2) + tid;

    float val = 0.f;
    if (gid < n)              val  = in[gid];
    if (gid + blockDim.x < n) val += in[gid + blockDim.x];

    val = warpReduceSum(val);                   // 第 1 级：Warp 内

    __shared__ float warp_results[32];
    if (lane == 0) warp_results[wid] = val;     // 只写 8 个值
    __syncthreads();

    if (wid == 0) {                             // 第 2 级：Warp 0 收尾
        val = (lane < (blockDim.x >> 5)) ? warp_results[lane] : 0.f;
        val = warpReduceSum(val);
    }
    if (tid == 0) out[blockIdx.x] = val;
}
```

为什么快：规约从「SMEM 往返 + 8 次屏障」变成「寄存器直通 + 1 次屏障」。SMEM 用量从 1024 B 降到 32 B。

7.10 V7：`float4` + Grid Stride，85%

```cpp
__global__ void reduce_v7(const float* in, float* out, int n) {
    int tid  = threadIdx.x;
    int lane = tid & 31;
    int wid  = tid >> 5;

    const float4* in4 = reinterpret_cast<const float4*>(in);
    int n4 = n >> 2;

    float val = 0.f;
    // grid-stride：步长 = 整个 grid 的线程数
    for (int i = blockIdx.x * blockDim.x + tid; i < n4;
         i += gridDim.x * blockDim.x) {
        float4 v = in4[i];
        val += (v.x + v.y) + (v.z + v.w);
    }

    val = warpReduceSum(val);
    __shared__ float wr[32];
    if (lane == 0) wr[wid] = val;
    __syncthreads();
    if (wid == 0) {
        val = (lane < (blockDim.x >> 5)) ? wr[lane] : 0.f;
        val = warpReduceSum(val);
    }
    if (tid == 0) out[blockIdx.x] = val;
}

// 启动：固定 grid = SM 数 × 4，和 n 解耦
int sms; cudaDeviceGetAttribute(&sms, cudaDevAttrMultiProcessorCount, 0);
int grid = sms * 4, block = 256;          // A100: 432 个 Block
```

两件事叠加：**每条指令搬 16 B 而不是 4 B**（指令数 ÷4），**grid 恰好填满 GPU**（无尾部空 Block）。

7.11 二次归约与工程落地

上面的 kernel 输出的是「每 Block 一个部分和」（长度为 grid 的数组），还需要第二次归约。三种做法：

做法	机制	适用
两段式 kernel	第二次 launch，把 grid 个值再规约	通用、可控
原子加	每个 Block 一次 `atomicAdd`	grid 小时够用
单 kernel + 计数器	最后一个 Block 收尾（第 6 章 fence 用法）	省一次 launch

工程上：**直接用 `cub::DeviceReduce::Sum`**。原文的结论是它通常能到 90%+ 带宽利用率，且维护成本为零。手写的价值在于「知道它在做什么」。

7.12 本章小结

版本	一句话记住它
V0	反面教材：分支 + bank conflict 双杀
V1	strided index 让发散只发生在尾部
V2	步长反转一举解决发散与冲突
V3	每线程多吃一点，减少 Block 数
V4	Warp 内不需要屏障，展开尾部
V5	编译期常量让死代码消失
V6	Shuffle 把 SMEM 往返变成寄存器直通
V7	向量化 + grid-stride 贴住带宽上限

第 8 章 经典算子之二：GEMM 的优化阶梯

8.1 为什么 GEMM 值得单独一章

深度学习的主干计算几乎都是 GEMM：全连接层、`im2col` 之后的卷积、Attention 的 QK^T 与 PV、MoE 的 expert 计算。GEMM 的优化史，基本就是 CUDA 性能优化史。

原文的优化阶梯有 8 级，每一级的收益都能用「算术强度」解释：

阶段	相对 cuBLAS（原文实测区间）	核心改进	算术强度
Naive	6%～11%	无	0.25
Shared Memory Tiling	25%～40%	数据复用	32（BM=BN=128）
Thread Tiling（寄存器外积）	50%～65%	减少 SMEM 访问	每线程级 4（TM=TN=8）
Vectorized Load（float4）	60%～75%	指令数 ÷4	—
Bank Conflict 消除	70%～80%	SMEM 吞吐回峰	—
Warp Tiling	80%～87%	Warp 内访问对齐	—
Double Buffering	90%～97%	隐藏 HBM 延迟	—
SASS 手调	95%～99%	寄存器调度 / reuse cache	—

【补充】cuBLAS 在大矩阵上通常能到理论峰值的 90%+，所以「相对 cuBLAS 97%」意味着「已经贴着硬件天花板」。

8.2 Naive：0.25 的算术强度

```
__global__ void sgemm_naive(const float* A, const float* B, float* C,
                            int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M || col >= N) return;
    float sum = 0.f;
    for (int k = 0; k < K; ++k)
        sum += A[row * K + k] * B[k * N + col];
    C[row * N + col] = sum;
}
```

算一个 `C[row][col]`：`2K` FLOPs，读 `A` 的一行 + `B` 的一列 = `8K` Bytes：

```
AI = 2K / 8K = 0.25 FLOP/Byte      ← 与 Reduce 同级，远低于脊点 156
```

而且 `A` 的同一行被 N 个线程重复读、`B` 的同一列被 M 个线程重复读——**复用率几乎为零，全部打到 HBM**。所以只有 6%～11%。

8.3 第一级：Block Tiling（25%～40%）

8.3.1 为什么沿 K 切

要算 `BM×BN` 的 C 块，理论上可以一次把 `BM×K` 和 `K×BN` 装进 SMEM。但 K 常常是几千到上万，两块加起来几十 MB，远超 SMEM（48～228 KB）。所以**沿 K 切成 BK 的小块，分批载入累加**。

沿 K 切还带来复用：每次载入的 `BM×BK`、`BK×BN` 会被整个 Block 的 `BM×BN` 个输出复用。

8.3.2 算术强度的推导

```
AI = (BM×BN×2×BK) / ((BM×BK + BK×BN) × 4)
   = BM×BN / (2×(BM+BN))
   = 1 / (2×(1/BM + 1/BN))
```

由均值不等式：在 `BM+BN`（SMEM 占用）固定时，**`BM=BN` 时 AI 最大**。取 `BM=BN=128`：

```
AI = 128×128 / (2×256) = 32 FLOP/Byte
```

比 naive 高 128 倍，但仍低于 A100 的脊点 156 → **还没到 Compute Bound，必须继续往下分块**。

8.3.3 协作加载要保证合并

`BM=128, BK=8, block=256` 时，加载 `tileA`（128×8）和 `tileB`（8×128）需要**不同的线程重排**：

```
int tid = threadIdx.x;

// 加载 A：8 列 → 让 8 个相邻线程负责一行，256 线程排成 32×8
int a_x = tid % 8, a_y = tid / 8;

// 加载 B：128 列 → 让 32 个相邻线程负责一行的连续段，排成 8×32
int b_x = tid % 32, b_y = tid / 32;

// 计算 C：16×16 网格，每线程负责 Tm×Tn = 8×8 个输出
int c_x = tid % 16, c_y = tid / 16;
```

📌 这种排布下，加载 B 时一个 Warp 恰好覆盖一行的 32 个连续 float（完美合并）；加载 A 时一个 Warp 覆盖连续 4 行 × 8 列。

8.4 第二级：Thread Tiling（50%～65%）——外积的威力

8.4.1 从内积到外积

```
// ❌ 内积写法：每次 FMA 都要读 A、B 各一个 float
for (int k = 0; k < K; ++k) sum += A[row][k] * B[k][col];

// ✅ 外积写法：一次读 TM + TN 个，做 TM×TN 次 FMA
for (int k = 0; k < K; ++k) {
    load a_frag[TM];        // TM 次 SMEM 读
    load b_frag[TN];        // TN 次 SMEM 读
    for (i in TM) for (j in TN)
        c_frag[i][j] += a_frag[i] * b_frag[j];   // TM×TN 次 FMA
}
```

8.4.2 收益量化

每轮 K 迭代：读 `TM+TN` 个 float（`4(TM+TN)` B），做 `2·TM·TN` FLOPs：

```
AI = 2·TM·TN / (4(TM+TN)) = TM·TN / (2(TM+TN))
```

`TM=TN=8` → **4**（按元素算是 64/16 = 4 倍复用）。原文的说法：原本 4 次 SMEM 访问才能驱动的计算，现在 1 次就够。

8.4.3 为什么是 8×8 而不是 16×16

组成	`TM=TN=8`	`TM=TN=16`
`c_frag`	64	256
`a_frag + b_frag`	16	32
总 float	80	288
实际寄存器/线程	~110～128	**溢出**（>255）

`16×16` 会让 `c_frag` 独占 256 个寄存器，直接 spill 到 local memory，**性能反而崩塌**。

`TM=TN=8` 时：`c_frag` 64 + `a/b_frag` 16 + 索引/临时 ~30 ≈ 110～128 寄存器 → V100 上一个 SM 放 2 个 Block（512 线程 = 16 Warp）→ **occupancy 25%**。

📌 原文对这个 25% 的注解很重要：**GEMM 有极高的 ILP，每线程 64 条独立 FMA 链，25% occupancy 完全够用**。这是「occupancy 不是越高越好」的最佳例证。

8.5 第三级：Warp Tiling（80%～87%）

8.5.1 为什么需要

Block 内的 8 个 Warp 如果不加约束，会出现「多个 Warp 抢同一块 SMEM 数据但访问模式不同」的问题。给每个 Warp 划分独立的 `32×64` 输出区域，访问模式就规整了。

8.5.2 形状怎么选：`4×8` 优于 `1×32`

```
AI_warp = 32·TM·TN·2 / ((x·TM + y·TN)·4)   , x·y = 32, TM=TN
        = 16·TM / (x + y)
```

要让 `x+y` 最小。`x·y=32` 的约束下，`x+y ≥ 2√32 ≈ 11.3`，整数解里 **`4×8`（x+y=12）和 `8×4` 最优**；`1×32`（x+y=33）最差——因为 Warp 内 32 个线程要么都要 A 的同一行（好）要么都要 B 的 32 个不同列（坏），总有一边吃亏。

8.5.3 坐标计算（背下来）

```
int warp_id = tid >> 5;              // tid / 32
int lane_id = tid & 31;              // tid % 32

// Block 内 8 个 Warp 排成 4×2
int warp_row = warp_id >> 1;         // 0..3
int warp_col = warp_id & 1;          // 0..1

// lane 在 Warp 内排成 4×8（行主序）
int lane_row = lane_id >> 3;         // 0..3
int lane_col = lane_id & 7;          // 0..7

// 线程负责的 8×8 子块起始坐标
int row_c = (warp_row * 4 + lane_row) * TM;
int col_c = (warp_col * 8 + lane_col) * TN;
```

验证：`tid=18` → `warp_id=0, lane=18, lane_row=2, lane_col=2` → `row_c=16, col_c=16`，负责第 16～23 行、16～23 列。

8.6 第四级：`float4` 向量化（60%～75%）

8.6.1 A 要转置存

问题：A 按行存进 SMEM 后，内层循环要按列读（同一行不同线程读同一列），无法用 `float4`。

解法：**SMEM 里存 `As[BK][BM]`（转置）**，这样原来的列访问变成行访问，可以 `float4` 读：

```
// 从全局读 A 的一行（连续），转置写进 SMEM
float4 v = *reinterpret_cast<const float4*>(&A[grow * K + bk + lc]);
As[lc + 0][lrow] = v.x;
As[lc + 1][lrow] = v.y;
As[lc + 2][lrow] = v.z;
As[lc + 3][lrow] = v.w;

// 从 SMEM 读时就是连续地址
float4 a4 = *reinterpret_cast<const float4*>(&As[k][thread_row]);
```

8.6.2 坐标公式要跟着改

`float4` 让每线程的 8 个元素从「间隔 16 行的 8 个离散位置」变成「两组各 4 个连续元素」：

```
tileC_row = 4 * C_THREAD_Y + (i / 4) * 4 * C_BLOCK_Y + (i % 4);
tileC_col = 4 * C_THREAD_X + (j / 4) * 4 * C_BLOCK_X + (j % 4);
```

【推断】这一步是最容易写错的地方——**向量化会改变线程与数据的映射关系**，不只是「把 4 次读合成 1 次读」。改完必须重新推导写回 C 的地址。

8.7 第五级：Bank Conflict 消除（70%～80%）

8.7.1 float4 下冲突只在 quarter-warp 内判定

访存宽度	Warp 字节	transaction	判定范围
32-bit	128 B	1	Warp（32）
64-bit	256 B	2	half-warp（16）
128-bit	512 B	4	quarter-warp（8）

所以 `4×8` warp 布局下，要看的是「8 个 lane 是否铺满一层 bank」。

8.7.2 三种方案

方案	做法	优点	缺点
Padding	`As[BK][BM + 4]`	简单	破坏 16 B 对齐，禁掉 `float4`
XOR Swizzle	`As[k][row ^ (k*4)]`	不浪费空间、保持对齐	索引复杂
拆 4×4 子块	每线程的 8×8 拆成 4 个 4×4	不改映射	写回变 scatter
Z-order	改 lane→(row,col) 映射	10 → 4 个 transaction	最难理解

8.7.3 Z-order 为什么是 4 个

```
int lane_row = lane_id % 2 + (lane_id / 16) * 2;   // 4 行
int lane_col = (lane_id % 16) / 2;                  // 8 列
```

tile	quarter-warp 内	跨 QW 合并	transaction
tileA	2 个地址，各 4 路广播	条件 B（xor 2）成立	2
tileB	4 个地址，各 2 路广播	条件 A（xor 1）成立	2
合计	—	—	**4**

对比基线 `1×8` 行主序的 10 个，**减少 60%**。

8.8 第六级：双缓冲（90%～97%）

前面所有版本都有一个共同问题：`LDG → STS → __syncthreads() → 计算` 是串行的，HBM 的 300～500 cycle 完全暴露。双缓冲把它变成流水线（详见 3.5 节）。

```
__shared__ float As[2][BK][BM];
__shared__ float Bs[2][BK][BN];
float a_frag[2][TM], b_frag[2][TN];
float c_frag[TM][TN] = {0.f};
int buf = 0;

load_global_to_shared(As[0], Bs[0], /*k=*/0);
__syncthreads();
load_shared_to_reg(As[0], Bs[0], a_frag[0], b_frag[0]);

for (int k = BK; k < K; k += BK) {
    load_global_to_shared(As[buf ^ 1], Bs[buf ^ 1], k);   // 先发 LDG
    compute(a_frag[buf], b_frag[buf], c_frag);            // 再算
    __syncthreads();
    buf ^= 1;                                             // 只 1 次屏障
    load_shared_to_reg(As[buf], Bs[buf], a_frag[buf], b_frag[buf]);
}
```

📌 双缓冲的额外好处：**主循环里只需要 1 次 `__syncthreads()`**，而不是 2 次。

8.9 第七级：`cp.async` 与 Hopper

Ampere 的 `cp.async` 省掉寄存器中转，可做 3～4 级流水；Hopper 的 TMA + `wgmma` 把「搬运」和「矩阵乘」彻底交给不同硬件单元。这也是 CUTLASS 的实际做法（Ampere 上默认 3～4 级 buffer，3 stage 通常是平衡点）。

再往下一级是 **SASS 级调优**（8.1 表里的最后一行）：寄存器 bank conflict（`reg_id % 4`）、`.reuse` 标记率（编译器约 20%，手调可达 49%+）、LDS 与 FFMA 指令交错。原文给的量级是「仅 reuse 标记率提升就能带来 5%～10%」，工具是 `cuobjdump` / `nvdisasm` 与 maxas/CuAssembler。这一级不建议初学者碰——它不可移植，且收益远小于前面七级。

8.10 参数表与「写法陷阱」

8.10.1 推荐参数

参数	推荐值	理由
BM, BN	128	AI=32，且 SMEM 放得下
BK	8	SMEM 可控（128×8×4 = 4 KB/矩阵）
TM, TN	8	寄存器 110～128，不 spill
Block	256	(128/8)×(128/8) = 16×16
Warp 形状	4×8	平衡 A/B 访问

参数	推荐值	理由
Registers/Thread	120～128	用 `__launch_bounds__` 控制
SMEM/Block	16～32 KB	含双缓冲

8.10.2 四个「等价但慢 30%」的写法

```
// ❌ 除法取模（非 2 的幂时是多条指令）
int r = tid / BN_TILE, c = tid % BN_TILE;
// ✅ 位运算
int r = tid >> 3, c = tid & 7;

// ❌ #pragma unroll 失效：起始值来自运行时变量，trip count 不可知
int start = threadIdx.x * TM;
#pragma unroll
for (int i = start; i < start + TM; ++i) { ... }
// ✅ 从 0 开始
#pragma unroll
for (int i = 0; i < TM; ++i) { ... }

// ❌ 只迭代 1 次的循环保留了循环结构
for (int k = 0; k < BK / 4; ++k) { ... }
// ✅ 手动展开

// ❌ 分块参数是运行时变量 → 每次都要算
// ✅ 作为 template 参数 → 编译期常量折叠
template <int BM, int BN, int BK, int TM, int TN>
__global__ void sgemm(...) { constexpr int TX = BN / TN; ... }
```

原文的结论很硬：**这套写法优化本身就能把性能从约 60% 推到约 93% cuBLAS**，因为它们减少的是「非计算指令」——整数运算、分支、地址计算占用了本该给 FFMA 的发射槽。

8.11 本章小结

```
Naive            AI=0.25        全走 HBM
  ↓ Block Tiling AI=32          复用 K 维
  ↓ Thread Tiling 外积          复用 SMEM
  ↓ float4        指令 ÷4       降发射压力
  ↓ Bank 消除     SMEM 回峰     10→4 transaction
  ↓ Warp Tiling   4×8 形状      对齐访问
  ↓ 双缓冲        重叠 LDG/FFMA  藏 300+ cycle
  ↓ cp.async/TMA  硬件流水      搬运与计算解耦
  ↓ Tensor Core   换计算单元    真正的数量级提升
```

一句话：**GEMM 的优化史就是「把数据搬得越来越少、搬得越来越早、算得越来越集中」的历史。**

第 9 章 经典算子之三：Softmax 的数值稳定与 Online 化

9.1 两篇文章解决两个不同问题

AIInfraGuide 把 Softmax 拆成两篇，因为它们回答的是**两个层次**的问题：

文章	问题	手段	最好成绩（A100，4096×4096）
5.1 朴素实现优化	「三遍扫描」太慢	并行化、Shuffle、向量化、融合	~75% 带宽（V4）
5.2 Online Softmax	「三遍」能不能变两遍/一遍	换算法（递推修正）	~82% 带宽（V3 One-Pass）

这是全模块唯一一次「算法层面的优化」压过「工程层面的优化」：前面 Reduce/GEMM 都是同一个算法换写法，这里换的是数学。

9.2 数值溢出：为什么必须减最大值

```
Softmax(x_i) = e^{x_i} / Σ e^{x_j}
```

精度	最大值	对应的 x 上限	超过会怎样
FP32	$\approx 3.4\times10^{38}$	≈88.7	`expf` 返回 `+inf`，全行变 NaN
FP16	$\approx 6.55\times10^{4}$	≈11.09	更快溢出

QK^T 未缩放时很容易超过 88（尤其是 `d_k` 大），所以朴素实现**必然崩**。

数学依据：减去同一常数不改变结果。

```
e^{x_i - c} / Σ e^{x_j - c} = (e^{x_i}·e^{-c}) / (Σ e^{x_j}·e^{-c}) = e^{x_i} / Σ e^{x_j}
```

取 `c = max(x)` 后所有指数参数 ⩽ 0，结果落在 (0,1]，上溢消除。代价是**多了一遍扫描求 max**。

9.3 三遍扫描与它的成本

遍次	操作	读
1	`m = max_j x_j`	N

遍次	操作	读
2	`d = Σ e^{x_j − m}`	N
3	`y_i = e^{x_i − m} / d`	N

总读 `3N`。Softmax 每元素 1～2 FLOP，是极端 Memory Bound，**少读一遍就是 33% 的加速**。

9.4 Safe Softmax 的五版演进

版本	核心	带宽（原文实测）	相对加速	关键点
V0	一线程一行	~5%	1.0x	并行度≈0，stride=N 非合并
V1	一 Block 一行 + SMEM 规约	~35%	7.0x	合并访存，但 8 次屏障
V2	Warp Shuffle 两级规约	~52%	10.4x	屏障 8 → 1
V3	`float4` 向量化	~65%	13.0x	指令数 ÷4
V4	两遍融合（SMEM 暂存 exp）	~75%	15.0x	少读一遍

9.4.1 V0 为什么只有 5%

```
__global__ void softmax_v0(float* in, float* out, int M, int N) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M) return;
    float* x = in + row * N;
    // 三遍串行扫完这一行
}
```

三个问题叠加：**每行只有 1 个线程**（4096 个元素串行）、**Warp 内 32 个线程访问 32 个不同行**（地址相差 `N*4` 字节，完全不合并）、**`expf` 算了两遍**。

9.4.2 V1/V2：把一行拆给整个 Block

```
__device__ float warpReduceMax(float v) {
    #pragma unroll
    for (int o = 16; o > 0; o >>= 1) v = fmaxf(v, __shfl_down_sync(0xffffffffu, v, o));
    return v;
}
__device__ float warpReduceSum(float v) {
    #pragma unroll
    for (int o = 16; o > 0; o >>= 1) v += __shfl_down_sync(0xffffffffu, v, o);
    return v;
}

// 两级：Warp 内 Shuffle → Warp 间 SMEM
__device__ float blockReduceMax(float v) {
    __shared__ float wmax[32];
    int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
    v = warpReduceMax(v);
    if (lane == 0) wmax[wid] = v;
    __syncthreads();
    int nw = blockDim.x >> 5;
    v = (lane < nw) ? wmax[lane] : -INFINITY;
    if (wid == 0) v = warpReduceMax(v);
    return v;                        // 结果在 tid 0，需要时再广播
}
```

【推断】注意 `warpReduceMax` 用 `fmaxf` 而不是 `max`，且初值用 `-INFINITY`——这两点在「全行都是负数」和「有 lane 不参与」时才不出错。

9.4.3 V4：两遍融合的边界

```
// Pass 1: 向量化 + Shuffle 求 max
// Pass 2: 计算 exp(x-m) 同时累加 d，并把 exp 值暂存进 SMEM
// 最后：从 SMEM 读 exp 值，除以 d，写回 out
extern __shared__ float smem[];   // 大小 = N，暂存 exp 值
```

对 `N=4096` 只需 16 KB SMEM，A100 上很宽裕。但：

N	SMEM 需求	可行性
4096	16 KB	✅
8192	32 KB	✅ 但 occupancy 下降
32768	128 KB	⚠️ 接近上限
131072	512 KB	❌ 放不下

N 一大就必须换算法，这正是 Online Softmax 的动机。

9.5 Online Softmax：用乘法修正代替重新计算

9.5.1 递推公式

维护 `(m_k, d_k)`，处理新元素 `x_k` 时：

```
m_{k+1} = max(m_k, x_k)
d_{k+1} = d_k · e^{m_k − m_{k+1}} + e^{x_k − m_{k+1}}
```

推导只有一步：原来累加的是 `e^{x_j − m_k}`，现在要变成 `e^{x_j − m_{k+1}}`，而

```
e^{x_j − m_{k+1}} = e^{x_j − m_k} · e^{m_k − m_{k+1}}
```

所以整个历史累加和只需乘一个修正因子 `e^{m_k − m_{k+1}}`。

💡 关键性质：**如果新元素不是新最大值，修正因子 = $e^0 = 1$**，退化为普通累加，零额外开销。

9.5.2 数值稳定性没被牺牲

性质	为什么成立
不会上溢	`m` 始终是已见元素最大值，所有指数参数 ⩽ 0
修正因子 ⩽ 1	因为 `m_{k+1} ≥ m_k`
结果与 Safe Softmax 等价	只有浮点结合顺序不同，无近似

9.6 并行化 Online Softmax：合并算子要满足结合律

单线程递推是串行的，并行化的前提是 **`(m, d)` 的合并满足结合律**：

```
m_merge = max(m1, m2)
d_merge = d1·e^{m1 − m_merge} + d2·e^{m2 − m_merge}
```

于是可以树形归约，也可以用 Shuffle：

```
__device__ void warpReduceOnline(float& m, float& d) {
    #pragma unroll
    for (int o = 16; o > 0; o >>= 1) {
        float m2 = __shfl_down_sync(0xffffffffu, m, o);
        float d2 = __shfl_down_sync(0xffffffffu, d, o);
        float mn = fmaxf(m, m2);
        d = d * __expf(m − mn) + d2 * __expf(m2 − mn);
        m = mn;
    }
}
```

9.6.1 一个真实的 NaN 陷阱

原文专门警告了这一点：第二级 Warp 规约时，若 `num_warps = 8`，而 `offset = 16`，lane 9 会从 lane 25 拿到 `m2 = -INFINITY`，此时：

```
m_new = max(-inf, -inf) = -inf
m - m_new = -inf - (-inf) = NaN
expf(NaN) = NaN  ⇒  整个 sum 变 NaN
```

正解：第二级只让 `lane < num_warps` 的线程参与，其余线程的 `m` 设成一个**有限**的哨兵值（如 `-FLT_MAX`），或者干脆用 `if (lane < num_warps)` 保护整段合并。

【推断】这是「教科书代码能跑、生产代码会挂」的典型：教科书假设 `num_warps = 32`，实际 `blockDim = 256` 时只有 8 个 Warp。

9.7 Online Softmax 的五版演进

版本	核心	扫描次数	带宽（原文实测）	相对加速
V0	一线程一行	2	~7%	1.0x
V1	Block 并行 + Shuffle 合并规约	2	~58%	8.3x
V2	`float4` 向量化	2	~70%	10.0x
V3	寄存器缓存 → One-Pass	1	**~82%**	11.7x
V4	Grid Stride 多行	2	~72%	10.3x

9.7.1 V3：真正的一遍（寄存器当缓存）

```
#define MAX_ELEMS_PER_THREAD 32

float reg[MAX_ELEMS_PER_THREAD];
float m = -INFINITY, d = 0.f;
int cnt = 0;

// 唯一一遍全局读：读的同时做 online 递推，并把值留在寄存器
for (int i = tid; i < N; i += blockDim.x) {
    float x = in[row * N + i];
    reg[cnt++] = x;
    float mn = fmaxf(m, x);
    d = d * __expf(m - mn) + __expf(x - mn);
    m = mn;
}
// 块内合并 (m, d) ...
// 第二遍：从寄存器取值，只写不读
for (int j = 0; j < cnt; ++j)
    out[row * N + tid + j * blockDim.x] = __expf(reg[j] - m) / d;
```

容量上限（原文表）：

blockDim	N 上限	每线程寄存器（仅缓存）
256	8192	32 × 4 B = 128 B
256	4096	16 × 4 B = 64 B
128	4096	32 × 4 B = 128 B

📌 **V3 在 N>8192 时会失效**（`MAX_ELEMS_PER_THREAD` 一大就 spill）。这就是为什么 V2（两遍）依然是「通用最优」。

9.7.2 V4：Grid Stride 的适用边界

V4 和 V2 算法完全一样，只改「Block ↔ 行」的映射（1:1 变 1:多）。实测 72%，**比 V2 还低**——因为本文的 M=4096 已经足够大，grid 调度开销不是瓶颈。

原文对这个「优化没收益」的解释很诚实，值得学：**Grid Stride 的收益在 M 极大时才体现**；而当 `M` 很小、`N` 很大（batch=1 的 decode）时，问题变成「Block 数不足」，需要的是「多个 Block 协作处理同一行 + 两段式归约」，而不是 grid-stride。

9.8 算子融合：Softmax 的终局

单独优化 Softmax 的收益是有上限的，因为真实推理里它从不单独出现：

```
未融合：Scale → Mask → Softmax → MatMul(V)
        4 次 kernel launch，3 次中间张量落地 HBM

融合后：一个 kernel 完成全部
        1 次 launch，0 次中间落地
```

融合模式	融合内容	收益来源
Bias + Activation	`y = act(xW + b)`	省一次读 + 一次写
LayerNorm + Dropout	归一化后立刻 dropout	省中间张量
Scale + Mask + Softmax	Attention 的常规三合一	省 2N 读写
Softmax + MatMul	FlashAttention 的核心	省整个 N×N 矩阵

【补充】三种融合实现路径的取舍：

路径	谁来做	优点	缺点
手写融合 kernel	人	性能上限最高	开发/维护成本高
`torch.compile`	TorchInductor	零改动	遇 Graph Break 退化
Triton	人（块级）	比 CUDA 快得多地写出融合	仍要懂分块

9.9 本章小结

概念	一句话
减最大值	数学等价，消除上溢，代价是多一遍
三遍 → 两遍	Online 递推用乘法修正历史
并行化前提	`(m,d)` 合并满足结合律
NaN 陷阱	`-inf - (-inf)` = NaN，哨兵值要用有限数
One-Pass	寄存器缓存换掉第二遍，受 N/blockDim 限制
终局	融合进 Attention，不再单独存在

第 10 章 Attention 算子：FlashAttention V1/V2 与 Decode 侧优化

10.1 标准 Attention 的两个瓶颈

```
S = QKᵀ/√d      (N×N)
P = softmax(S)  (N×N)
O = PV          (N×d)
```

指标	量级	含义
中间矩阵 S、P	$O(N^2)$	`N=4096, d=128` 时单个 head 的 S 就要 32 MB（FP16）
HBM 读写	$O(N^2)$	反复读写 N×N
计算量	$O(N^2d)$	两次矩阵乘
算术强度	$\approx O(d)$	**d 只有 64/128，远低于脊点**

所以标准 Attention 是 **Memory Bound**，不是 Compute Bound。原文的比喻很准：厨师做菜很快，但食材从仓库搬到厨房太慢，厨师大部分时间在等食材。

【推断】这解释了一个反直觉的事实：**FlashAttention 减少了 FLOPs 吗？没有，它甚至因为重计算多做了 FLOPs。它减的是 HBM 访问。** 在 Memory Bound 的场景下，「多算少搬」是净赚。

10.2 FlashAttention V1：Tiling + Online Softmax

10.2.1 两个技术缺一不可

技术	解决什么
Tiling	如何不生成完整的 N×N 矩阵
Online Softmax	分块计算时，Softmax 依赖全局信息怎么办

10.2.2 块大小怎么定

```
Bc = ceil(M / (4d)),   Br = min(ceil(M / (4d)), d)
```

`M` 是 SRAM 容量（以元素计）。分母是 `4d` 而不是 `d`，因为一个块计算时 SRAM 要同时驻留 `Q_i, K_j, V_j, O_i` 四块 `B×d` 的张量，合计约 `4Bd`，要求 `4Bd ≤ M`。

`Br` 再和 `d` 取 min，是原论文为简化分析加的约束（`Br ≤ d`），V2 放宽了。

SRAM 内容	大小
`Q_i`	`Br × d`
`K_j`	`Bc × d`

SRAM 内容	大小
`V_j`	`Bc × d`
`S_ij`	`Br × Bc`
`O_i`	`Br × d`
`m_i, l_i`	`2Br`

主导项是 `(2Br + 2Bc) × d`，这就是 `4d` 的由来。

10.2.3 前向的五个步骤

```
Step 1  S_ij = Q_i K_jᵀ / √d
Step 2  m̃_ij = rowmax(S_ij)
Step 3  m_new = max(m_i, m̃_ij)
Step 4  α = e^{m_i − m_new}
        l_new = α·l_i + rowsum(e^{S_ij − m_new})
Step 5  O_new = (α·l_i / l_new)·O_i + (1/l_new)·e^{S_ij − m_new}·V_j
```

10.2.4 V1 的循环结构（也是它的病根）

```
外循环 j：遍历 K/V 块
    加载 K_j, V_j 到 SRAM（每块只加载一次）
    内循环 i：遍历 Q 块
        加载 Q_i, O_i, m_i, l_i
        算 S_ij、更新统计量、更新 O_i
        把 O_i, m_i, l_i 写回 HBM      ← 每个 j 都写一次！
```

这带来两个后果：

1. `O` 被读写 `2Tc` 次（`Tc` 次读 + `Tc` 次写）；
2. 不同 Q 块共享统计量，寄存器装不下，必须落到 HBM（辅助张量 `M`、`L`）。

10.2.5 V1 的并行度瓶颈（最关键的一条）

```
grid = (batch, head)
```

推理场景常常 `batch=1, heads=32` → **只有 32 个 program**，而 A100 有 108 个 SM、H100 有 132 个 SM：

```
SM 利用率 ≈ 32 / 108 ≈ 30%
```

⚠️ 这是 V1 最大的短板，也是 V2 必然出现的动因。**注意这个瓶颈和「代码写得好不好」无关，是并行划分的粒度问题。**

10.3 IO 复杂度：为什么说它「渐近最优」

10.3.1 标准 Attention

```
HBM 访问 = O(Nd + N²) = O(N²)          (N ≫ d 时 N² 主导)
```

10.3.2 FlashAttention

```
HBM 访问 = O(N²d² / M)
```

拆解：

步骤	结论
块大小约束 $Bc = O(M/d)$	外循环次数 $Tc = N/Bc = O(Nd/M)$
每次外循环遍历完整 Q 和 O（共 $O(Nd)$）	Q/O 总访问 $O(N^2d^2/M)$
K/V 整体只读一次	$O(Nd)$，可忽略

因为典型 $d^2=16384$ 远小于 M（H100 的 228 KB SMEM ≈ 114K 个 FP16），N^2d^2/M 比 N^2 小两个数量级以上。

论文还证明：**任何精确 Attention 算法的 HBM 下界就是 $\Omega(N^2d^2/M)$**，所以 FA 在渐近意义上已经最优。

10.3.3 显存收益

项	标准 Attention	FlashAttention
中间矩阵	$O(N^2)$	**O(N)**（只存 m、l）
反向所需	存 S、P	重计算

原文的实测量级（A100）：

序列长度	标准	FA V1	加速比
1024	1.0x	2.4x	2.4x
2048	1.0x	2.8x	2.8x
4096	1.0x	3.5x	3.5x
8192	OOM	可运行	—

10.4 反向传播：用重计算换显存

标准反向需要 S、P（N×N）。FA 的选择是**不存，重算**：

保存什么	前向只存 `Q, K, V, O, m, l`
怎么恢复 P	`P_ij = diag(l_i)^{-1} · e^{S_ij − m_i}`
代价	额外约 50% 前向计算量
为什么划算	Attention 是 Memory Bound，多算能被少搬完全覆盖

一个关键恒等式（V1 论文附录 B.3 提出）：

```
rowsum(dP ⊙ P) = rowsum(dO ⊙ O) =: D_i
```

左式要对长度 N 的整行做归约（P、dP 可能装不进 SRAM），右式只需对长度 d 的向量做点积——**代价与序列长度无关**。这是反向能塞进 SRAM 的前提。

10.5 FlashAttention V2：三个改动 + 一个附加

10.5.1 V1 只跑到 25%～40%

问题	原因	影响
循环顺序不佳	外循环 K/V → O 反复读写、Q 重复加载	额外 HBM 流量 + 无法跨 Q 并行
非 GEMM 运算占比高	Softmax rescaling 大量逐元素操作	Tensor Core 空转
Warp 切分维度不当	沿 K/V 切 → 多 Warp 共写同一行 O	频繁 Warp 间同步

10.5.2 改动一：换循环顺序（外 Q 内 KV）

```
外循环 i：遍历 Q 块（并行到不同 Thread Block）
    加载 Q_i 到 SRAM（只一次）
    初始化 O_i = 0, m_i = -inf, l_i = 0
    内循环 j：遍历 K/V 块
        加载 K_j, V_j
        算 S_ij，更新统计量与 O_i（全在 SRAM）
    把最终 O_i 写回 HBM（只一次）
```

三重收益：

收益	机制
O 只写一次	HBM 访问从 `2Tc` 降到 1

收益	机制
外循环天然并行	不同 Q 块完全独立 → grid 可以到 `N/Br`
Q 只加载一次	整个内循环驻留 SRAM

📌 原文的洞察：**「减少 O 的读写」比「减少 K/V 的读」收益更大**，因为 O 的读写次数正比于外循环数。

10.5.3 改动二：延迟 rescaling（减非 GEMM 运算）

V1（每块都归一化）	V2（只维护未归一化累积）
`O = (l_old·α/l_new)·O + (1/l_new)·e^{S−m}V`	`Õ = α·Õ + e^{S−m}V`
每轮 2 次除法 + 1 次乘法	每轮只做指数缩放
—	最后统一 `O = Õ/l`

省掉的正是「反归一化 → 重新归一化」这套来回折腾。`Tc` 越大，省得越多。

10.5.4 改动三：Warp 沿 Q 切（不沿 KV 切）

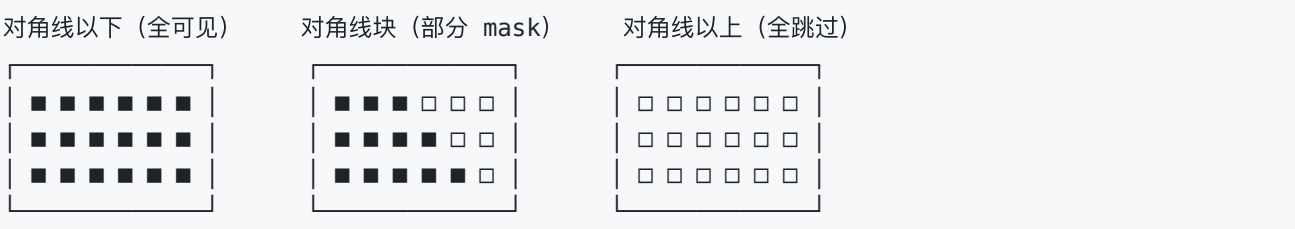
❌ 沿 KV 切（V1）	✅ 沿 Q 切（V2）
多 Warp 需合并部分 Softmax 结果	每 Warp 独立完成 Softmax
需要 SMEM / shuffle 通信	无需 Warp 间同步
O 的同一行被多 Warp 共写	O 的不同行由不同 Warp 独占

这就是「split-K 需要 reduce」vs「split-M 无需 reduce」的区别。

10.5.5 附加：Causal 块级跳过

对第 `i` 个 Q 块和第 `j` 个 KV 块：

位置关系	处理
完全在对角线下方（全可见）	正常计算，**不套 mask**
完全在对角线上方（全不可见）	**直接跳过**
跨对角线	逐元素套 mask



对 Causal Attention 可减少约 **50%** 的无效计算。

10.5.6 保守量：存 logsumexp

V2 前向保存 `L_i = m_i + log(l_i)` 而不是分别存 `m`、`l`：

- 统计量存储减半；
- 反向直接 `P_ij = exp(S_ij - L_i)`，省一次除法。

10.5.7 V1 vs V2 总表

方面	V1	V2
外循环	K/V 块	Q 块（并行）
O 的 HBM 访问	`2Tc`	1
内循环归一化	2 除 + 1 乘	只做指数缩放
Warp 切分	沿 K/V（需通信）	沿 Q（无通信）
Causal 优化	无	块级跳过
保存统计量	`m`、`l`	`L = m + log l`

10.5.8 实测（A100，原文表）

序列长度	V1 TFLOPS	V2 TFLOPS	V2 利用率
1024	124	196	63%
2048	136	218	70%
4096	141	227	73%
8192	138	222	71%

V2 相比 V1 约 **1.6x**；利用率从 25%～40% 提到 50%～73%。

与其他实现对比（A100-80GB，N=2048，d=128）：

实现	前向	反向
PyTorch 标准	1.0x	1.0x
FlashAttention V1	2.8x	2.5x
FlashAttention V2	**4.3x**	**3.9x**
xFormers (cutlass)	3.1x	2.8x

10.6 V3、Flash-Decoding、PagedAttention（原文仅给概述）

⚠️ **口径说明**：模块二的第 6 章只有 6.1/6.2 两篇正文，V3、Flash-Decoding、FlashDecoding++、FlashInfer、PagedAttention 只在**章节简介**里给了定位描述，没有展开正文。下面的细节属于【补充】，只保证机制正确，不引用原文数据。

10.6.1 FlashAttention-3（Hopper 专属）

原文简介的定位：利用 Hopper 特性（WGMMA、TMA、FP8）做异步流水线。

【补充】三个关键手段：

手段	作用
`wgmma` + TMA 生产者/消费者 warp 分工	搬运与计算彻底解耦（warp specialization）
GEMM 与 Softmax 两级流水交错（ping-pong）	让 Tensor Core 与 CUDA Core 同时忙
FP8 分块量化 + 误差补偿	在保持精度的前提下翻倍算力

【量级】论文口径：H100 上 FP16 前向可达理论峰值的约 75%，FP8 更高；相对 FA2 的加速在 1.5～2.0x 区间（具体以论文为准）。

10.6.2 Flash-Decoding：decode 阶段并行度不足

问题：decode 时 `q` 只有 1 个 token，**没有 Q 维可并行**；grid 只剩 `batch × heads`，又一次掉进 V1 的坑。

【补充】解法是「沿 KV 维切分 + 两段式归约」：

```
Step 1  把 KV 切成 S 段，每段独立算局部 (m, l, O)   ← 并行度 ×S
Step 2  用 logsumexp 把 S 个局部结果合并             ← 极轻量
```

合并公式与 Online Softmax 的 `(m,d)` 合并完全同构——**这套数学在模块里出现了三次：Softmax、FA 前向、decode 归约。**

`S` 的取值要满足 `batch × heads × S ≥ SM 数 × 每 SM 驻留数`，典型取 64～128 段【推断】。

10.6.3 PagedAttention：KV Cache 的分页管理

原文简介的定位：vLLM 中「虚拟页 → 物理页」映射在 GPU 上的实现。

【补充】核心机制：

概念	类比	作用
Block（固定大小，如 16 token）	内存页	KV Cache 的分配单位
Block Table	页表	逻辑块 → 物理块的映射

概念	类比	作用
按需分配	缺页分配	消除预分配的内部碎片

【量级】vLLM 论文报告显存浪费从传统预分配的 60%～80% 降到 4% 以下。kernel 侧的代价是：注意力计算时要做一次「block table 查表 + gather」，**访存不再连续**，需要用额外的索引张量把分块的 KV 拼起来。

10.7 本章小结

结论	依据
Attention 是 Memory Bound	AI ≈ O(d) ≪ 脊点
FA 不减 FLOPs，减 HBM	IO 复杂度 $O(N^2d^2/M)$ 且已达下界
V1 的病根是并行度	grid = batch×head，常只有几十
V2 的钥匙是换循环	O 写一次 + 外循环天然并行
V2 的加速是 1.6x	25–40% → 50–73%
V3/decode 优化	原文只给概述，细节属补充

第 11 章 AI 编译器：Triton、torch.compile、TVM/XLA

11.1 为什么需要「不手写 CUDA」

手写 CUDA 的性能上限最高，但开发成本也最高：一个高性能 GEMM 要几千行，改一个 shape 就要重新调参。AI 编译器的定位是**在开发效率和性能之间取一个点**。

方案	抽象层级	谁负责线程/warp	性能上限（量级）	开发成本
CUDA	thread 级	人	100%（基准）	极高
Triton	tile/block 级	编译器	手写 CUDA 的 80%～95%	中
torch.compile	图级	编译器（生成 Triton）	视算子而定	极低
TVM / XLA	图级 + 调度	编译器（自动搜索）	视硬件而定	低

11.2 Triton 的 block-level 编程模型

11.2.1 心智模型对照

CUDA 概念	Triton 概念	说明
thread	—（不存在）	程序员不写线程
thread block	**program instance**	由多个 warp 协作，处理一个或多个 tile
`blockIdx`	`tl.program_id(axis)`	最多 3 维
`gridDim`	`tl.num_programs(axis)`	—
`blockDim`（线程数）	`num_warps`（启动参数）	一个 program 用几个 warp
`float4` 手工向量化	编译器自动	由访问模式推断
`__shared__` 手工分配	编译器自动	由 tile 大小推断
`__syncthreads()`	编译器自动	不暴露给用户

11.2.2 六个核心 API

```
@triton.jit
def kernel(x_ptr, y_ptr, n, BLOCK: tl.constexpr, num_stages: tl.constexpr):
    pid  = tl.program_id(0)                 # 类似 blockIdx.x
    npg  = tl.num_programs(0)               # 类似 gridDim.x
    offs = tl.arange(0, BLOCK)              # 块内偏移，BLOCK 必须是 2 的幂
    mask = offs < n
    x = tl.load(x_ptr + offs, mask=mask, other=0.0)   # 越界处填 0.0
    y = tl.exp(x)                                       # 逐元素
    s = tl.sum(y, axis=0)                               # 块内规约
    tl.store(y_ptr + offs, y, mask=mask)
```

API	作用	易错点
`tl.program_id`	取 program 编号	不是线程编号
`tl.arange`	构造连续偏移	长度必须是 2 的幂
`tl.load/store`	块级读写	**必须给 `other`**，否则 mask 处是未定义值
`tl.where`	逐元素选择	**两个分支都会被计算**，不能当 `if` 用
`tl.dot`	块级矩阵乘	走 Tensor Core，是性能关键
`tl.range(..., num_stages=k)`	软件流水线	级数越高，SMEM/寄存器压力越大

11.2.3 Fused Softmax 骨架

```
@triton.jit
def _softmax(in_ptr, out_ptr, in_stride, out_stride, n_rows, n_cols,
             BLOCK_SIZE: tl.constexpr, num_stages: tl.constexpr):
    row_start = tl.program_id(0)
    row_step  = tl.num_programs(0)         # persistent：一个 program 处理多行
    for row in tl.range(row_start, n_rows, row_step, num_stages=num_stages):
        cols = tl.arange(0, BLOCK_SIZE)
        mask = cols < n_cols
        p = in_ptr + row * in_stride + cols
        row_data = tl.load(p, mask=mask, other=-float('inf'))   # 关键：-inf
        row_minus_max = row_data - tl.max(row_data, axis=0)
        numerator = tl.exp(row_minus_max)
        denominator = tl.sum(numerator, axis=0)
        softmax_out = numerator / denominator
        tl.store(out_ptr + row * out_stride + cols, softmax_out, mask=mask)
```

【推断】`other=-float('inf')` 是这段代码能正确的关键：补齐位置的输入若是 0，`exp(0-m)` 会污染分母；用 `-inf` 才能保证 `exp(-inf)=0`。这和 CUDA 里手动 mask 是同一个坑。

11.3 torch.compile：Dynamo + Inductor

```
PyTorch 代码 → TorchDynamo（字节码级图捕获）→ TorchInductor（代码生成）→ Triton kernel
```

11.3.1 Graph Break：编译的最大敌人

Dynamo 遇到「无法静态捕获」的操作会打断计算图，退化回逐算子执行：

触发原因	例子	规避
Python 副作用	`print`、写全局变量	移出被编译函数
数据依赖的控制流	`if x.sum() > 0:`	改成 `torch.where`
不支持的算子	某些自定义 op	注册 `custom_op` 或 `allow_in_graph`
动态 shape 变化	每步变长	`dynamic=True` 或 pad 到固定长度

排查：

```
TORCH_LOGS="graph_breaks" python train.py     # 列出每处打断及原因
TORCH_LOGS="recompiles"  python train.py     # 看是否反复重编译
```

【推断】Graph Break 的代价不是「变慢一点」，而是「融合全部失效」——两个相邻算子一旦被拆到两个图里，就无法合成一个 kernel，中间张量必须落地 HBM。

11.4 TVM / XLA 的定位

编译器	核心	强项	弱项
TVM	张量表达式 + 调度原语 + autoTVM/Ansor 自动调优	跨硬件（x86/ARM/GPU/加速器）	手写调度的学习曲线
XLA	JIT + 全图融合 + 布局优化	JAX/TF 生态、TPU	对动态 shape 不友好
TorchInductor	生成 Triton	PyTorch 生态零改动	依赖 Triton 后端
Triton	块级 DSL	自定义算子	需要人写分块

一句话区分：**Triton 解决「怎么写一个快 kernel」，torch.compile 解决「怎么少写 kernel」，TVM/XLA 解决「怎么跨硬件生成 kernel」。**

11.5 本章小结

结论	一句话
Triton 的抽象	tile 级，编译器管线程/warp/SMEM
Triton 仍要管的	分块、grid、occupancy 预算、mask
融合是主收益	少 launch、少落地，N 越大越明显
Graph Break	编译优化失效的头号原因
选型	自研算子用 Triton，整图用 torch.compile

第 12 章 性能分析工具链：nsys / ncu / torch.profiler

12.1 三个工具的分工

原文第 8 章（性能分析工具链）只给了章节简介，未展开正文；本章的命令与指标表属于【补充】，与原文的定位描述一致。

工具	粒度	回答什么问题	典型命令
Nsight Systems	时间线（μs～s）	GPU 为什么有空隙？CPU 在等什么？	`nsys profile -o t ./app`

工具	粒度	回答什么问题	典型命令
Nsight Compute	单 kernel（cycle）	这个 kernel 撞哪堵墙?	`ncu --set full ./app`
PyTorch Profiler	算子级	哪个 op 最耗时?	`torch.profiler.profile(...)`

顺序不能反：先 nsys 找「哪个 kernel 值得优化」，再 ncu 下钻，最后 torch.profiler 对账（框架层是否还有多余 op）。

12.2 Nsight Systems：宏观时序

```
nsys profile -o trace --trace=cuda,nvtx,osrt ./train
nsys stats trace.nsys-rep                  # 汇总
nsys-ui trace.nsys-rep                     # 图形界面
```

看什么	判据	结论
GPU 时间线上有长空隙	空隙处 CPU 在跑	**CPU 瓶颈**（dataloader/预处理）
空隙处有 `cudaMemcpy`	传输未与计算重叠	用 pinned memory + 多 stream
空隙处有 `NCCL` kernel	通信等待	通信/计算未 overlap
大量极短 kernel 连排	launch overhead	融合或 CUDA Graph
kernel 之间空隙 <1μs	正常	不是瓶颈

【补充】两个必备技巧：① 用 NVTX 打标记把「一个 iteration」框出来，否则时间线太长看不到重点：

```
torch.cuda.nvtx.range_push("step")
# ... forward / backward ...
torch.cuda.nvtx.range_pop()
```

② `--cuda-memory-usage=true` 可以同时看显存分配曲线。

12.3 Nsight Compute：Kernel 级下钻

```
ncu --set full -o k ./app                          # 全量（很慢，慎用）
ncu --set detailed ./app                           # 常用
ncu --kernel-name regex:sgemm --launch-count 3 ./app
ncu --metrics sm__throughput.avg.pct_of_peak_sustained_active,\
dram__throughput.avg.pct_of_peak_sustained_active ./app
ncu --import k.ncu-rep --page details              # 回看
ncu --import k1.ncu-rep --import k2.ncu-rep --page diff   # 两版本对比
```

12.3.1 SOL（Speed of Light）面板怎么读

指标	含义	目标
Compute (SM) Throughput	计算单元利用率	Compute Bound 时应 >70%
Memory Throughput	显存带宽利用率	Memory Bound 时应 >70%
Achieved Occupancy	实际活跃 Warp 占比	>25%（非越高越好）
Warp Execution Efficiency	分支发散程度	>85%
L2 Hit Rate	L2 命中率	>70%
Bank Conflicts	共享内存冲突	0

判读顺序：先看两个 Throughput 谁高 → 定 Memory/Compute Bound；再看 Occupancy Limiters → 定资源瓶颈；再看 Warp State（stall 原因）→ 定具体动作。

12.3.2 stall 原因对照动作

Stall 名称	含义	该做什么
Long Scoreboard	等全局内存	双缓冲、提 occupancy、向量化
Short Scoreboard	等共享内存	查 bank conflict、减少 LDS 次数
MIO Throttle	SMEM 指令排队过多	减少 LDS/STS，用寄存器复用
Barrier	等 `__syncthreads()`	减少屏障、展开尾部 Warp
Not Selected	Warp 太多抢发射槽	不是问题（说明并行度足够）
Math Pipe Throttle	算术单元打满	Compute Bound，换 Tensor Core 或降精度

12.4 PyTorch Profiler：框架层对账

```
from torch.profiler import profile, ProfilerActivity, tensorboard_trace_handler

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             schedule=torch.profiler.schedule(wait=1, warmup=1, active=3),
             on_trace_ready=tensorboard_trace_handler("./log"),
             record_shapes=True, with_stack=True) as prof:
    for step in range(6):
        train_step()
        prof.step()

print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=15))
```

列	用途
`Self CUDA Time`	该 op 自身 kernel 时间（排除子调用）
`CUDA total`	含子调用的总时间
`# Calls`	调用次数（次数多而时间短的，是融合候选）
`CPU Time` vs `CUDA Time`	CPU 远大于 CUDA → CPU 瓶颈

【补充】`record_shapes=True` + `with_stack=True` 是排「谁调用了这个 op」的唯一实用手段，代价是 trace 变大。

12.5 本章小结

工具	一句话
nsys	先看「哪段没在算」，再谈优化
ncu	两个 Throughput 定性质，stall 表定动作
torch.profiler	对账：框架层有没有多余 op
方法论	测量 → 归因 → 单变量改动 → 复测

第 13 章 与已有笔记的衔接

这套 M1 笔记不是孤立的，它和你已经写过的四份笔记是「同一批概念的不同投影」。对照着读，效率最高。

已有笔记	对应本笔记	可以复用的部分	需要重新学的部分
CSAPP 学习笔记	第 3 章存储层次	Cache line、局部性、组相联、写回/写直达	「bank」在 GPU 是 4 B 宽的 32 路并行 SRAM，不是 cache 的 set

已有笔记	对应本笔记	可以复用的部分	需要重新学的部分
CMU 15–418 学习笔记	第 4、5 章	并行模式、work distribution、强/弱扩展	GPU 用「多 Warp 轮转」代替乱序执行与深流水
CS149 并行计算速查	第 4、7 章	归约、scan、SIMD 与分支	CUDA 的 SIMT 允许 lane 独立寻址，SIMD 不允许
6.172 性能工程速查	第 8、12 章	Roofline、优化循环、profiler 驱动	GPU 的 roofline 有两条「屋顶」（CUDA Core / Tensor Core）
ECE408 应用并行编程（若已有）	第 2、3、7 章	CUDA 线程层级、tiling、合并访存	本笔记补上了「为什么」的微架构解释
CS336 学习笔记	第 9、10 章	Attention 数学、KV Cache、训练显存	这里是 kernel 视角：IO 复杂度、重计算、并行划分

13.1 三条「迁移学习」的捷径

1. **存储层次只学一遍，用两套语言描述。** CSAPP 的语言是 cache line / 组相联 / 命中率；GPU 的语言是 sector / coalescing / bank conflict。第 3 章那张表就是翻译字典。
2. **并行模式上移一层，成本模型不变。** 15–418 讲线程级归约，第 7 章讲 Block 级归约，第 10 章讲跨 SM 的 decode 归约——三者都是「树形 + 两段式」，成本都是「通信次数 × 单次延迟 + 数据量 / 带宽」。
3. **性能工程的方法论完全通用。** 6.172 的「测量 → 建模 → 优化 → 复测」就是第 12 章的诊断剧本；区别只是 GPU 多了一个「Warp 调度」的自由度。

13.2 一个反例：不要照搬 CPU 的直觉

CPU 直觉	GPU 现实	原因
「分支预测能藏住分支」	分支直接串行化	SIMT 无分支预测，靠 mask
「Cache 越大越好」	L1 和 SMEM 抢同一块 SRAM	物理共享
「减少线程数能减少开销」	线程几乎免费，Warp 才是成本	上下文切换零开销
「访存延迟靠预取解决」	靠「多 Warp + 双缓冲」解决	无硬件预取器（有，但不可控）
「循环展开总是好的」	展开过度 → 寄存器溢出 → 更慢	255 寄存器硬上限

第 14 章 复习清单与自测题

14.1 复习清单（按「能不能上手写」排序）

#	能力	对应章节	自检方式
1	手算任意配置下的 `gid` 与越界判断	2	给 `grid=100, block=128, n=10000`，说出哪些线程越界
2	用 CUDA event 计时并换算有效带宽	2	写出「读 2 写 1」的带宽公式
3	判断一段访存是否合并	3	看 `stride` 与 `threadIdx` 的关系
4	手算 SMEM 的 bank 号并判断冲突路数	3	`tile[32][32]` 转置为何 32-way
5	用 padding / swizzle 消冲突	3	写出 `[32][33]` 的 bank 推导
6	写 `float4` 版本并处理尾部	3	头对齐 + 尾标量
7	画双缓冲的 LDG/FFMA/STS 时序	3	说明为何「先 LDG 后 FFMA」
8	识别并消除 Warp 发散	4	把 `tid%2` 改成 `tid/32%2`
9	写 `__shfl_down_sync` 的 sum/max 归约	4	5 步、mask=0xffffffff
10	从寄存器/SMEM/Block 三维手算 occupancy	5	复现 5.3 的例题
11	解释「25% occupancy 的 GEMM 为何快」	5	ILP vs TLP
12	用 `__launch_bounds__` 换 occupancy 并验证	5	看 `-Xptxas -v` 前后对比
13	写正确的 `__syncthreads()` 与 fence 收尾	6	复现 6.4 的 last-block 逻辑
14	把「N 线程抢 1 地址」改成分层归约	6	三层：Warp→SMEM→Global
15	写完 Reduce V2 并解释两个瓶颈同时消失	7	步长反转的双重收益
16	写完 Reduce V6（Shuffle 两级）	7	只需 1 次屏障
17	推导 GEMM tiling 后 AI=32	8	`BM·BN/(2(BM+BN))`
18	解释外积为何比内积省 SMEM 访问	8	`TM·TN/(2(TM+TN))`
19	推出 `4×8` warp 形状的最优性	8	均值不等式
20	写 Safe Softmax 与 Online Softmax	9	三遍 vs 两遍
21	推导 `(m,d)` 合并公式并说明结合律	9	FlashAttention 的数学基础
22	说清 FA V1 的并行度瓶颈与 V2 的三处改动	10	grid 只有 batch×heads
23	推导 FA 的 IO 复杂度	10	$O(N^2d^2/M)$
24	用 Triton 写 fused softmax 并算 occupancy	11	`other=-inf` 的坑

#	能力	对应章节	自检方式
25	用 nsys 找 GPU 空隙、用 ncu 定 Memory/Compute Bound	12	两个 Throughput 的判读

14.2 自测题（12 题，答案在后）

Q1. 一个 kernel 用 256 线程/Block、每线程 64 个寄存器、每 Block 32 KB SMEM，在 A100 上 occupancy 是多少？瓶颈是谁？

Q2. `smem[tid] += smem[tid + 64]`（`blockDim=256`，`tid<64`）会不会有 bank conflict？

Q3. 为什么 Reduce 的 V1 已经消掉了大部分发散，却只从 15% 提到 32%？

Q4. `float4` 加载下，一个 Warp 的 128-bit 访问被拆成几个 transaction？冲突在多少线程内判定？

Q5. GEMM 里 `BM=BN=128, BK=8` 的分块，算术强度是多少？够不够饱和 A100？

Q6. 为什么 `TM=TN=16` 的线程分块反而比 `8×8` 慢？

Q7. Online Softmax 的修正因子 `e^{m_old - m_new}` 在什么情况下等于 1？这说明什么？

Q8. 为什么 Online Softmax 的第二级 Warp 规约容易出现 NaN？怎么修？

Q9. FlashAttention V2 把外循环从 KV 换成 Q，收益有哪三条？

Q10. 为什么 FlashAttention 多做了 FLOPs（重计算）却更快？

Q11. Triton 的 `tl.where(den != 0, num/den, 0.0)` 有什么问题？

Q12. ncu 显示 `Memory Throughput 88%`、`Compute 15%`、`Bank Conflicts 0`，下一步该做什么？

14.3 答案

A1. 寄存器：`64×32 = 2048`/Warp → `65536/2048 = 32` Warp。SMEM：`164/32 = 5` Block = 40 Warp。Block 数上限 32 不触顶；线程上限 64 Warp。取 `min(32,40,64) = 32` Warp → 对齐到 8 Warp/Block 的整数倍仍是 32 → **occupancy = 50%**，瓶颈是**寄存器**。

A2. 不会。`tid` 从 0 到 63 连续，起始 bank 覆盖 0～31 两轮，但每一个 32 线程的 Warp 内（如 tid 0～31）访问 `tid` 与 `tid+64`，`(tid+64) % 32 = tid % 32`——同一个线程的两次访问落在同一 bank，**但不同 lane 之间不冲突**（lane i 与 lane j 的 bank 不同）。补充：这两次访问是「同一线程的两条指令」，不在同一 transaction 内判定，所以无冲突。

A3. 因为 V1 只解决了发散，**bank conflict 反而更严重**（`s=1` 时 2-way、`s=2` 时 4-way、`s=4` 时 8-way）。SMEM 一直是串行化的，所以带宽只到 32%。

A4. 拆成 **4 个** transaction（Warp 512 B / 128 B），冲突只在 **quarter-warp（8 个 lane）** 内部判定；此外 `xor 1`/`xor 2` 的 lane 对若访问同一地址，可触发跨 quarter-warp 的 uniform 合并，使请求数从 4 降到 2。

A5. `AI = BM·BN/(2(BM+BN)) = 128×128/(2×256) = 32 FLOP/Byte`。A100 脊点约 156，**差 5 倍，不够**——所以必须继续做 Thread/Warp 级分块，把 SMEM 访问也复用起来。

A6. `c_frag` 需要 `16×16 = 256` 个寄存器，加上 `a_frag/b_frag`（32）和索引（~30），总量超过 255 的硬上限 → **寄存器溢出到 local memory（本质是显存）**，每次访问都变成一次全局访存，性能崩塌。

A7. 当新元素**没有刷新最大值**（`m_new = m_old`）时，因子 = `e^0 = 1`。说明**绝大多数迭代的修正开销为零**，Online 的额外成本只在「刷新最大值」这个稀疏事件上付出。

A8. 第二级规约里，若 `lane > num_warps`，该 lane 的 `m` 是 `-INFINITY`；与另一条 `-INFINITY` 合并时 `m_new = -inf`，而 `m - m_new = -inf - (-inf) = NaN`，`expf(NaN)=NaN` 污染整个和。**修法**：用有限哨兵（`-FLT_MAX`），或用 `if (lane < num_warps)` 保护，或先广播再只在有效 lane 上合并。

A9. ① `O` 的 HBM 访问从 `2Tc` 降到 1 次；② 外循环（Q 块）天然独立，可直接映射到不同 Block，解决 V1「grid 只有 batch×head」的并行度瓶颈；③ `Q_i` 只加载一次并常驻 SRAM 整个内循环。

A10. 因为 Attention 是 **Memory Bound**：瓶颈在 HBM 搬运而不在计算。重计算多花的是 FLOPs（而且 Tensor Core 有空闲），省下的是 `O(N²)` 的中间矩阵读写——**省下的时间远多于多花的时间**（原文：额外约 50% 前向计算量，换来显存 `O(N²)→O(N)`）。

A11. `tl.where` **两个分支都会求值**，`num/den` 在 `den==0` 时已经执行了除法，可能产生 `inf`/`NaN` 并触发异常或污染结果。正确写法是先 `safe_den = tl.where(den != 0, den, 1.0)`，做除法，再用 `tl.where` 选回 0。

A12. 已经是 **Memory Bound 且访存效率接近上限**（88%）、也没有 bank conflict。此时继续「优化访存」的边际收益很小，应该：① 先确认 `dram__bytes` 是否等于理论最小搬运量（若大于，说明有冗余读写 → 考虑融合）；② 若已是最小搬运量，则**优化到此为止**，转而看上层（减少调用次数、融合相邻算子），或换更快的存储路径（如 `__ldg`、L2 持久化）。

14.4 三条「如果只记三件事」

1. **先判性质，再谈手段**：Memory Bound 就减搬运，Compute Bound 就换单元，Latency Bound 就提并行度。搞错性质，所有优化都是白工。
2. **一切优化都在「减少搬运次数」和「隐藏搬运延迟」之间**：Tiling/融合/复用属于前者，双缓冲/cp.async/多 Warp 属于后者。
3. **测量优先于直觉**：每一版优化都要有 profiler 数据和「为什么有效」的机制解释，否则你只是碰巧改对了一次。

第 15 章 参考来源

15.1 AIInfraGuide 模块二（24 篇，访问日期 2026-11）

章节	篇目	本笔记主要用在
快速入门	CUDA 编程快速入门指南	第 1、2、11、12 章（roofline 表、错误检查宏、ncu 指标阈值）
第 1 章	第1章-cuda编程入门 / 1.1 开发环境搭建 / 1.2 编程模型 / 1.3 内存模型 / 1.4 第一个实用 kernel	第 2、3 章
第 2 章	第2章-cuda性能优化基础 / 2.1 Warp 与执行模型 / 2.2 内存访问优化 / 2.3 Occupancy 与资源分配 / 2.4 同步与原子操作	第 4、5、6 章
第 3 章	3.1 CUDA Reduce 算子优化	第 7 章（V0～V7 全部数据）
第 4 章	4.1 CUDA GEMM 算子性能优化	第 8 章（优化阶梯、transaction 分析、参数表）
第 5 章	5.1 Softmax 朴素实现优化 / 5.2 Online Softmax 实现	第 9 章
第 6 章	6.1 FlashAttention V1 详解 / 6.2 FlashAttention V2 详解	第 10 章（V3/Flash-Decoding/PagedAttention 仅来自章节简介）
第 7 章	7.1 Triton 编程框架介绍	第 11 章
第 8 章	第8章-性能分析工具链	第 12 章（**仅有章节简介，无正文**）

页面前缀：`https://caomaolufei.github.io/AIInfraGuide/cuda/模块二-cuda编程与算子优化/`

15.2 补充来源

来源	用途
NVIDIA, *CUDA C++ Programming Guide*	SIMT、`__shfl_*_sync` 语义、`cp.async`、内存一致性、`__launch_bounds__`
NVIDIA, *CUDA C++ Best Practices Guide*	Occupancy、Memory Optimizations、指令吞吐表
Dao et al., *FlashAttention* (NeurIPS 2022)	IO 复杂度、下界证明、重计算、`D_i` 恒等式
Dao, *FlashAttention-2* (2023)	循环顺序、work partitioning、Causal 跳过
Milakov & Gimelshein, *Online normalizer calculation for softmax* (2018)	Online Softmax 递推

来源	用途
Triton 官方教程（Fused Softmax / Matrix Multiplication）	block-level 模型、`num_warps` / `num_stages`
NVIDIA Nsight Compute / Nsight Systems 用户手册	SOL 面板、stall 名称、metric 命名
NVIDIA A100 / H100 架构白皮书	SM 资源、SMEM 容量、Tensor Core 规格
CUTLASS 文档	多级流水线、swizzle 布局

15.3 免责与口径重申

- 本笔记中所有百分比性能数字，**均转述 AIInfraGuide 原文在 A100 80GB SXM4 上的实测**，不是本笔记复现的结果；换硬件后绝对值会变。
- 所有硬件规格（SM 数、SMEM、带宽、TFLOPS）取自 NVIDIA 公开规格与原文表格，**B200 的稠密/稀疏口径差异**见 1.3 节说明。
- 标注【补充】的段落（尤其是 FA V3、Flash-Decoding、PagedAttention、第 8 章工具链细节）**不对应原文正文**，请以官方文档/论文为准。
- 若发现本笔记与 AIInfraGuide 原文冲突，以原文为准。